

INTRODUCTION TO KRYLOV SUBSPACE METHODS FOR THE SOLUTION OF LINEAR SYSTEMS

L. Giraud
INRIA- Bordeaux Sud Ouest
joint CERFACS-INRIA Lab
and
S. Gratton
ENSEEIHT-IRIT, Toulouse

2ème Année Informatique
Majeure Mathématique
2009-2010

Contents

1	Reliability of the calculation	5
1.1	Backward error	5
1.2	Sensitivity v.s. conditioning	6
1.3	Backward stability of algorithms	9
2	Algorithm selection	10
3	Why searching solutions in Krylov subspaces	12
4	Unsymmetric Krylov solvers based on the Arnoldi procedure	13
4.1	The FOM variants	15
4.2	The GMRES variants	17
4.3	Strategies for restarted GMRES	28
5	Symmetric Krylov solvers based on the Lanczos approach	32
5.1	Lanczos algorithm for linear systems	33
5.2	The conjugate gradient algorithm	35
6	The Lanczos biorthogonalization approach	39
6.1	The BiCG algorithm	42
6.2	The QMR algorithm	44
6.3	Transpose free variant	45
6.3.1	The CGS algorithm	45
6.3.2	The BiCGStab algorithm	47
7	Algebraic preconditioning techniques	50
7.1	Driving principles	50
7.2	Preconditioned CG	51
7.3	Preconditioner taxonomy and location	52
7.4	Some classical algebraic preconditioners	54
7.5	Spectral preconditioners	57
7.6	Flexible solver	59
8	Bibliography	61

1 Reliability of the calculation

How to assess the quality of a computed solution ?

Relative norm should be preferred. Let x and $\tilde{x}$ be the solution and its approximation, $\rho = \frac{\|x - \tilde{x}\|}{\|x\|}$ gives the number of correct digits in the solution. Example: $e = 2.7182818\dots$

Approximation	ρ
2.	$2 \cdot 10^{-1}$
2.7	$6 \cdot 10^{-3}$
2.71	$3 \cdot 10^{-3}$
2.718	$1 \cdot 10^{-4}$
2.7182	$3 \cdot 10^{-5}$
2.71828	$6 \cdot 10^{-7}$

How to assess the quality of a computed solution ?

If $\frac{\|x - \tilde{x}\|}{\|x\|}$ is large, **TWO** possible reasons:

- the mathematical problem is sensible to perturbations
- the selected numerical algorithm behaves poorly in finite precision calculation

The backward error analysis (Wilkinson, 1963) gives a framework to look at the problem.

1.1 Backward error

Backward error and conditioning

The backward error analysis, introduced by Wilkinson (1963), is a powerful concept for analyzing the quality of an approximate solution:

1. it is independent of the details of round-off propagation: the errors introduced during the computation are interpreted in terms of perturbations of the initial data, and the computed solution is considered as exact for the perturbed problem;
2. because round-off errors are seen as data perturbations, they can be compared with errors due to numerical approximations (consistency of numerical schemes) or to physical measurements (uncertainties on data coming from experiments for instance).

Backward error

Rigal and Gaches result (1967): the normwise backward error

$$\eta_{A,b}^N(\tilde{x}) = \min\{\varepsilon : (A + \Delta A)\tilde{x} = b + \Delta b, \|\Delta A\| \leq \varepsilon\|A\|, \|\Delta b\| \leq \varepsilon\|b\|\} \quad (1)$$

is given by

$$\eta_{A,b}^N(\tilde{x}) = \frac{\|r\|}{\|A\|\|\tilde{x}\| + \|b\|} \quad (2)$$

where $r = b - A\tilde{x}$.

Backward error

Exercise 1. Show that $\frac{\|r\|}{\|A\|\|\tilde{x}\| + \|b\|}$ is an lower-bound of $\eta_{A,b}^N(\tilde{x})$

The bound is attained for $\Delta A_{min} = \frac{\|A\|}{\|\tilde{x}\|(\|A\|\|\tilde{x}\| + \|b\|)} r \tilde{x}^T$

and

$$\Delta b_{min} = -\frac{\|b\|}{\|A\|\|\tilde{x}\| + \|b\|} r.$$

We have $\Delta A_{min} \tilde{x} - \Delta b_{min} = r$ with $\|\Delta A_{min}\| = \frac{\|A\|\|r\|}{\|A\|\|\tilde{x}\| + \|b\|}$ and $\|\Delta b_{min}\| = \frac{\|b\|\|r\|}{\|A\|\|\tilde{x}\| + \|b\|}$.

Normwise backward error

We can also define

$$\begin{aligned} \eta_b^N(\tilde{x}) &= \min\{\varepsilon : A\tilde{x} = b + \Delta b, \|\Delta b\| \leq \varepsilon\|b\|\} \\ &= \frac{\|r\|}{\|b\|} \end{aligned}$$

- Classically $\eta_{A,b}^N$ or η_b^N are considered to implement stopping criterion in iterative methods.
- Notice that $\frac{\|r_k\|}{\|r_0\|}$ (often seen in some implementations) reduces to η_b^N if $x_0 = 0$.

Componentwise backward error

Using similar techniques but componentwise perturbations we obtain:

$$\begin{aligned} \eta_{A,b}^C(\tilde{x}) &= \min\{\varepsilon : (A + \Delta A)\tilde{x} = b + \Delta b, |\Delta A| \leq \varepsilon|A|, |\Delta b| \leq \varepsilon|b|\} \\ &= \max_{i=1,\dots,n} \frac{|A\tilde{x} - b|_i}{(|A|\|\tilde{x}\| + |b|)_i}, \end{aligned}$$

with convention that $\frac{0}{0} = 0$.

This backward error enables to better cope with problems with large scaling.

1.2 Sensitivity v.s. conditioning

Sensitivity v.s. conditioning

Let suppose that we have solved $(A + \Delta A)(x + \Delta x) = b + \Delta b$. We would like to know how $\frac{\|\Delta x\|}{\|x\|}$ depends on $\frac{\|\Delta A\|}{\|A\|}$ and $\frac{\|\Delta b\|}{\|b\|}$. We denote $\omega = \max\left\{\frac{\|\Delta A\|}{\|A\|}, \frac{\|\Delta b\|}{\|b\|}\right\}$.

$$\begin{aligned} (A + \Delta A)(x + \Delta x) &= b + \Delta b \\ \Rightarrow \Delta x &= -A^{-1}\Delta Ax - A^{-1}\Delta A\Delta x + A^{-1}\Delta b \\ \Rightarrow \|\Delta x\| &\leq \|A^{-1}\| \frac{\|\Delta A\|}{\|A\|} \|A\|\|x\| + \|A^{-1}\| \frac{\|\Delta A\|}{\|A\|} \|A\|\|\Delta x\| \\ &\quad + \|A^{-1}\| \|b\| \frac{\|\Delta b\|}{\|b\|} \\ \Rightarrow \|\Delta x\| &\leq \omega \kappa^N(A) \|x\| + \kappa^N(A) \omega \|\Delta x\| + \omega \|A^{-1}\| \|b\| \\ \Rightarrow (1 - \omega \kappa^N(A)) \|\Delta x\| &\leq \omega \kappa^N(A) \|x\| + \omega \|A^{-1}\| \|b\| \end{aligned}$$

Sensitivity v.s. conditioning (cont)

Exercise 2. Show that, under appropriate assumptions, the following relation holds

$$\frac{\|\Delta x\|}{\|x\|} \leq 4\omega \kappa^N(A)$$

that reads

$$\text{Relative Forward error} \lesssim (\text{Condition number}) \times (\text{Backward error})$$

Sensitivity v.s. conditioning (cont)

- $\kappa^N(A, b) = \kappa^N(A) + \frac{\|A^{-1}\| \|b\|}{\|x\|}$ is the condition number of the linear system $Ax = b$ because (??) is attainable to first order in ω .
- $\kappa^N(A) = \|A^{-1}\| \|A\|$ is the condition number of the matrix A .
- More general definition of condition number exists based on Frechet derivative.

Sensitivity v.s. conditioning (cont)

Similar results can be established using componentwise perturbations

- $\kappa^C(A, b) = \frac{\| |A^{-1}| (|A| |x| + |b|) \|_\infty}{\|x\|_\infty}$ is the componentwise condition number of the linear system $Ax = b$.
- $\kappa^C(A) = \frac{\| |A^{-1}| |A| \|_\infty}{\|x\|_\infty}$ is the componentwise condition number of the matrix A .

Sensitivity v.s. conditioning (cont)

$$\begin{pmatrix} 1 & 1 \\ 1.000005 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2.000005 \end{pmatrix}$$

$\Downarrow$

$$\begin{pmatrix} 1 & 1 \\ 1.000006 & 1 \end{pmatrix} \begin{pmatrix} 0.83333 \dots \\ 1.16666 \dots \end{pmatrix} = \begin{pmatrix} 2 \\ 2.000005 \end{pmatrix}$$

First order estimate of the forward error

Matrix	$\kappa^N(A)$	$\eta_{A,b}^N$	$\kappa^N(A) \times \eta_{A,b}^N$	FE
M1	1 10 ²	1 10 ⁻¹⁶	1 10 ⁻¹⁴	2 10 ⁻¹⁵
M2	4 10 ⁵	9 10 ⁻¹⁷	3 10 ⁻¹¹	8 10 ⁻¹²
M3	1 10 ¹⁶	2 10 ⁻¹⁶	2	6 10 ⁻¹
M4	3 10 ¹	3 10 ⁻⁷	2 10 ⁻⁵	9 10 ⁻⁶
M5	4 10 ¹	6 10 ⁻⁴	3 10 ⁻²	2 10 ⁻²
M6	2 10 ¹²	7 10 ⁻¹⁷	1 10 ⁻⁴	4 10 ⁻¹³

Algorithm: Gauss elimination with partial pivoting (MATLAB).

Matrices : M1 - augment(20) M2 - dramadah(2) M3 - chebvand(20)
M4 - will(40) M5 - will(50) M6 - dd(20)

First order estimate of the forward error (cont)

M6 is a badly scaled matrix.

Matrix	$\kappa^C(A)$	$\eta_{A,b}^C$	$\kappa^C(A) \times \eta_{A,b}^C$	FE
M6	3	3 10 ⁻¹³	9 10 ⁻¹³	4 10 ⁻¹³

First order estimate of the forward error (cont)

- Large backward error: unreliable algorithm $\Rightarrow$ change the algorithm (ex. use Gauss elimination with pivoting).
- Ill-conditioned matrices: large condition number $\Rightarrow$ scale the matrix, $\Rightarrow$ think about alternative for the numerical formulation of the problem.

More computation ...

Computation of $I_n = \int_0^1 x^n e^{-x} dx$, $n > 0$

By part integration $I_0 = 1 - 1/e$, $I_n = nI_{n-1} - 1/e$, $n > 1$

Computation with 16 significative digits: $\tilde{I}_{100} = 5.7 \cdot 10^{+141}$

Backward recurrence: $I_{300} = 1(??)$, $I_{n-1} = \frac{1}{n} \left(I_n + \frac{1}{e} \right)$

Computation with 16 significative digits: $\tilde{I}_{100} = 3.715578714528098... \cdot 10^{-3}$

Exact value: $\tilde{I}_{100} = 3.715578714528085... \cdot 10^{-03}$

The art to select the good algorithm !!

How to solve a linear system

Solve the linear system $Ax = b$ where $A \in \mathbb{R}^{n \times n}$ is a nonsingular matrix and x and b are two vectors of $\mathbb{R}^n$.

Even though $x = A^{-1}b$, it does not make sense to compute the inverse of $A \Rightarrow$ too expensive, unreliable.

$$\begin{aligned} A &= \text{diag}\{k^{-i/(n-1)}\}, \quad i = 0, \dots, n-1 \\ A &= Q^*AQ, \quad x = (1, \dots, 1)^T, \quad b = A * x \end{aligned}$$

For $n = 50$ and $k = 10^{12}$: $\kappa^N(A) = k = 10^{12}$

	$\eta_N(\tilde{x})$	# flops
$\tilde{x} \leftarrow A^{-1}b$	$1.7 \cdot 10^{-6}$	268925
$\tilde{x} \leftarrow ??$	$1.3 \cdot 10^{-16}$	103450

How to solve a linear system

Example A : $A(i, i)$ is null $\Rightarrow$ division by zero

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Exemple B : $A(i, i)$ small $\Rightarrow$ numerical difficulties

$$A = \begin{pmatrix} 10^{-4} & 1 \\ 1 & 1 \end{pmatrix} \quad b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Assume we have a 3 digit arithmetic. The solution is

$$x = \left(\frac{10^4}{10^4 - 1}, \frac{10^4 - 2}{10^4 - 1} \right)^T$$

that rounds to

$$\text{fl}(x) = (1, -1)^T.$$

How to solve a linear system (cont)

Factorisation

$$L = \begin{pmatrix} 1 & 0 \\ 10^4 & 1 \end{pmatrix} \Rightarrow \text{fl}(L) = L$$

$$U = \begin{pmatrix} 10^{-4} & 1 \\ 0 & 1 - 10^4 \end{pmatrix} \Rightarrow \text{fl}(U) = \bar{U} = \begin{pmatrix} 10^{-4} & 1 \\ 0 & -10^4 \end{pmatrix}$$

Solution

$$Ly = b \Rightarrow y = (1, \quad 2 - 10^4)^T; \text{fl}(y) = \bar{y} = (1, \quad -10^4)^T$$

$$\bar{U}x = \bar{y} \Rightarrow \bar{x} = (0, \quad 1)^T$$

instead of $(1, \quad 1)^T$ but, $\kappa^N(A) \sim 2.6$

$\eta_{A,b}^N \sim 0.26 \gg 10^{-4}!!$

How to solve a linear system (cont)

Permute the rows of A and b .

$$A = \begin{pmatrix} 1 & 1 \\ 10^{-4} & 1 \end{pmatrix} \quad b = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Factorization:

$$L = \begin{pmatrix} 1 & 0 \\ 10^{-4} & 1 \end{pmatrix} \Rightarrow \text{fl}(L) = L$$

$$U = \begin{pmatrix} 1 & 1 \\ 0 & 1 - 10^{-4} \end{pmatrix} \Rightarrow \text{fl}(U) = \bar{U} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

Solution

$$Ly = b \Rightarrow y = (2, \quad 1 - 2 \cdot 10^{-4})^T; \text{fl}(y) = \bar{y} = (2, \quad 1)^T$$

$$\bar{U}x = \bar{y} \Rightarrow x = (1, \quad 1)^T$$

The result is correct for 3 digit calculation.

1.3 Backward stability of algorithms

Backward stability of Gaussian elimination with partial pivoting

Let $\tilde{x}$ the solution computed by Gaussian elimination with partial pivoting.

Then $(A + \Delta A)\tilde{x} = b$ with $\frac{\|\Delta A\|_\infty}{\|A\|_\infty} \leq 8n^3 \rho \psi + \mathcal{O}(\psi^2)$ where ψ is the machine precision.

ρ is the *growth factor*; $\rho = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|}$.

If ρ is large it might imply numerical instability (i.e. large backward error)

Backward stability of GMRES with robust orthogonalization

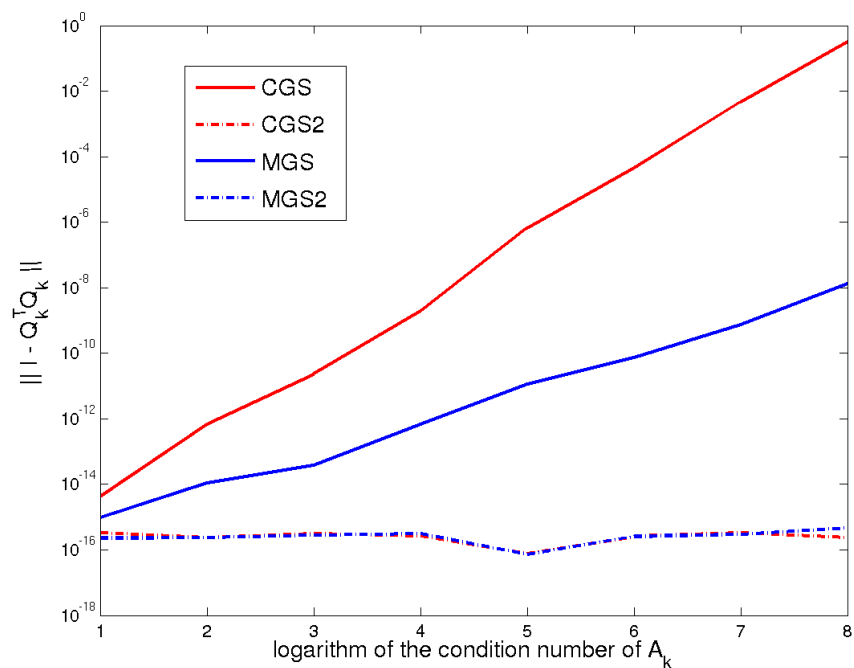
For not too ill-conditioned matrix, the Householder GMRES implementation ensures that

$$\eta_{A,b}^N(\tilde{x}_n) = \frac{\|b - A\tilde{x}_n\|}{\|A\|\|\tilde{x}\| + \|b\|} \leq \mathcal{P}(n)\psi + \mathcal{O}(\psi^2)$$

[J. Drkošová, M. Rozložník, Z. Strakoš and A. Greenbaum, *Numerical stability of the GMRES method*, BIT, vol. 35, p. 309-330, 1995.]

[C.C. Paige, M. Rozložník and Z. Strakoš, *Modified Gram-Schmidt (MGS), Least Squares, and Backward Stability of MGS-GMRES*, SIAM J. Matrix Anal. Appl., vol. 28 (1), p. 264-284, 2006.]

Backward stability of GMRES with robust orthogonalization



2 Algorithm selection

Direct and/or iterative ?

Algorithm selection

Two main approaches:

- direct solvers: compute a factorization and use the factors to solve the linear system; that is, express the matrix A as the product of matrices having simple structures (i.e. diagonal, triangular).
Example: LU for unsymmetric matrices, LL^T (Cholesky) for symmetric positive definite matrices, LDL^T for symmetric indefinite matrices.
- iterative solvers: build a sequence $(x_k) \rightarrow x^*$.

Solution of linear systems $Ax = b$ (Direct or Iterative approaches ?)

Direct methods

- Very general techniques
 - High numerical precision
 - Sparse matrices with irregular patterns
- Factorization of matrix A
 - May be costly in terms of flops or memory for building the factors
 - Allow to compute solutions corresponding to multiple right-hand sides b
- Computing issues
 - Good granularity of computations
 - Several levels of parallelism can be exploited

Iterative methods

- Efficiency depends on the type of problem
 - convergence – preconditioning
 - numerical properties or structure of A
- Require the product of A by a vector
 - less costly in terms of memory and possibly flops
 - computations of solutions with multiple right-hand sides can be prohibitive
- Computing issues
 - Smaller granularity of computations
 - Often, only one level of parallelism

Algorithm selection (II)

Matrix	Algorithm	Complexity
general	LU	$2/3n^3$
SPD	Cholesky	$n^3/3$
Banded	banded Chol.	$\mathcal{O}(n^2)$
Sparse	CG	$\mathcal{O}(n^{3/2})$
PDE	Multigrid	$\mathcal{O}(n)$

Direct solvers

Why libraries should be preferred to numerical recipes ? Solution of an SPD linear system

Size	500	1000	1500
Num. Recip.	2.9	30.2	106
BLAS 1	0.4	12.0	45
BLAS 3	0.2	1.4	5

Elapsed time (sec) on a SGI 02K

Sparse matrices

The effect of the ordering

$$A = \begin{pmatrix} x & x & x & x \\ x & x & & \\ x & & x & \\ x & & & x \end{pmatrix} = \begin{pmatrix} x & & & \\ x & x & & \\ x & x & x & \\ x & x & x & x \end{pmatrix} \begin{pmatrix} x & x & x & x \\ & x & x & x \\ & & x & x \\ & & & x \end{pmatrix}$$

while

$$PAP^T = \begin{pmatrix} x & & & x \\ & x & & x \\ & & x & x \\ x & x & x & x \end{pmatrix} = \begin{pmatrix} x & & & \\ & x & & \\ & & x & \\ x & x & x & x \end{pmatrix} \begin{pmatrix} x & & & x \\ & x & & x \\ & & x & x \\ & & & x \end{pmatrix}$$

$\Rightarrow$ symbolic analysis.

When pivoting is required : e.g. the unsymmetric case

Trade-off between

- preserve sparsity,
- preserve stability (pivot selection).

Stationary v.s. Krylov methods

Alternative to direct solvers when memory and/or CPU constraints. Two main approaches

- Stationary/asymptotic method:

$$x_k = f(x_{k-1})$$

with $x_k \rightarrow x^* \forall x_0$.

- Krylov method:

$$x_k = x_0 + \text{span}\{r_0, Ar_0, \dots, A^{k-1}r_0\}$$

with $r_k = b - Ax_k$ subject to some constraints/optimality conditions.

Basic scheme

Let x_0 be given and $M \in \mathbb{R}^{n \times n}$ a nonsingular matrix, compute

$$x_k = x_{k-1} + M(b - Ax_{k-1}). \quad (3)$$

Note that $b - Ax_{k-1} = A(x_* - x_{k-1}) \Rightarrow$ the best M is A^{-1} .

Theorem 1. *The stationary scheme defined by (3) converges to $x^* = A^{-1}b$ for any x_0 iff $\rho(I - MA) < 1$, where $\rho(I - MA)$ denotes the spectral radius of the iteration matrix $(I - MA)$.*

Some well-known schemes

Depending on the choice of M we obtain some of the best known stationary methods. Let decompose $A = L + D + U$, where L is lower triangular part of A , U the upper triangular part and D is the diagonal of A .

- $M = I$: Richardson method,
- $M = D^{-1}$: Jacobi method,
- $M = (L + D)^{-1}$: Gauss-Seidel method.

Notice that M has always a special structure and inverse must never be explicitly computed. $z = M^{-1}y$ reads *solve the linear system $Mz = y$* .

3 Why searching solutions in Krylov subspaces

Krylov method : some background



Aleksei Nikolaevich Krylov

1863-1945: Russia, Maritime Engineer

His research spans a wide range of topics, including shipbuilding, magnetism, artillery, mathematics, astronomy, and geodesy. In 1904 he built the first machine in Russia for integrating ODEs.

In 1931 he published a paper on what is now called the "Krylov subspace".

Definition 1. Let $A \in \mathbb{R}^{n \times n}$ and $r \in \mathbb{R}^n$ the space denoted by $\mathcal{K}(r, A, m)$ (with $m \leq n$) and defined by

$$\mathcal{K}(r, A, m) = \text{Span}\{r, Ar, \dots, A^{m-1}r\}$$

is referred to as the Krylov space of dimension m associated with A and r .

Why using this search space ?

For the sake of simplicity of exposure, we often assume $x_0 = 0$. This does not mean a loss of generality, because the situation $x_0 \neq 0$ can be transformed with a simple shift to the system

$$Ay = b - Ax_0 = \bar{b},$$

for which obviously $y_0 = 0$. The minimal polynomial $q(t)$ of A is the unique monic polynomial of minimal degree such that $q(A) = 0$. It is constructed from the eigenvalues of A as follows. If the distinct eigenvalues of A are $\lambda_1, \dots, \lambda_\ell$ and if λ_j has index m_j (the size of the largest Jordan block associated with λ_j), then the sum of all indices is

$$m = \sum_{j=1}^{\ell} m_j, \text{ and } q(t) = \prod_{j=1}^{\ell} (t - \lambda_j)^{m_j}. \quad (4)$$

When A is diagonalizable m is the number of distinct eigenvalues of A ; when A is a Jordan block of size n , then $m = n$.

Exercise 3. Show that $A^{-1} = -\frac{1}{\alpha_0} \sum_{j=0}^{m-1} \alpha_{j+1} A^j$.

Taxonomy of the Krylov subspace approaches

The Krylov methods for identifying $x_m \in \mathcal{K}(A, b, m)$ can be distinguished in four classes:

- **The Ritz-Galerkin approach:** construct x_m such that $b - Ax_m \perp \mathcal{K}(A, b, m)$.
- **The minimum norm residual approach:** construct $x_m \in \mathcal{K}(A, b, m)$ such that $\|b - Ax_m\|_2$ is minimal
- **The Petrov-Galerkin approach:** construct x_m such that $b - Ax_m$ is orthogonal to some other m -dimensional subspace.
- **The minimum norm error approach:** construct $x_m \in A^T \mathcal{K}(A, b, m)$ such that $\|b - Ax_m\|_2$ is minimal.

Constructing a basis of $\mathcal{K}(A, b, m)$

- The obvious choice $b, Ab, \dots, A^{m-1}b$ is not very attractive from a numerical point of view since the vectors $A^j b$ becomes more and more colinear to the eigenvector associated with the dominant eigenvalue. In finite precision calculation, this leads to a loss of rank of this set of vectors. Suppose that A is diagonalizable $A = VDV^{-1}$ and denote $\underline{b}$ the component of b in V . In V , $A^m b \equiv V D^m \underline{b}$.
- A better choice is to use the Arnoldi procedure.

4 Unsymmetric Krylov solvers based on the Arnoldi procedure

Arnoldi

Walter Edwin Arnoldi 1917-1995: USA. His main research subjects covered vibration of propellers, engines and aircraft, high speed digital computers, aerodynamics and acoustics of aircraft propellers, lift support in space vehicles and structural materials.

"The principle of minimized iterations in the solution of the eigenvalue problem" in Quart. of Appl. Math., Vol.9 in 1951.

The Arnoldi procedure

This procedure builds an orthonormal basis of $\mathcal{K}(A, b, m)$.

ARNOLDI'S ALGORITHM [-10pt]

- 1: $v_1 = b/\|b\|$
- 2: **for** $j = 1, 2, \dots, m-1$ **do**
- 3: Compute $h_{i,j} = v_i^T A v_j$ for $i = 1, \dots, j$
- 4: Compute $w_j = A v_j - \sum_{i=1}^j h_{i,j} v_i$
- 5: Compute $h_{j+1,j} = \|w_j\|$
- 6: Exit if $(h_{j+1,j} = 0)$
- 7: Compute $v_{j+1} = w_j/h_{j+1,j}$
- 8: **end for**

The Arnoldi procedure properties

Proposition 4.1. *If the Arnoldi procedure does not stop before the m^{th} step, the vectors $v_1, \dots, v_m$ form an orthonormal basis of the Krylov subspace $\mathcal{K}(A, b, m)$*

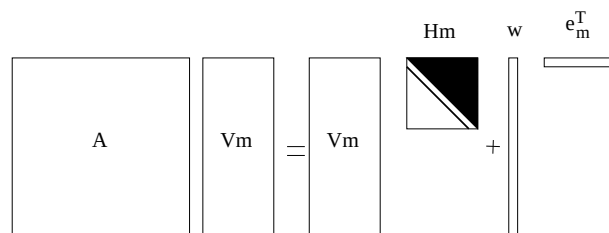
Exercise 4. Give a proof of the proposition 4.1.

Proposition 4.2. Denote V_m the $n \times m$ matrix with column vector $v_1, \dots, v_m$; $\bar{H}_m$ the $(m+1) \times m$ Hessenberg matrix whose nonzero entries are h_{ij} and by H_m the square matrix obtained from $\bar{H}_m$ by deleting its last row. Then the following relations hold:

$$A V_m = V_m H_m + h_{m+1,m} v_{m+1} e_m^T \quad (5)$$

$$= V_{m+1} \bar{H}_m \quad (6)$$

$$V_m^T A V_m = H_m \quad (7)$$



Exercise 5. Give a proof for equality (5).

Exercise 6. Give a proof for equality (6).

Exercise 7. Give a proof for equality (7).

Arnoldi for linear system

This technique belongs to the [Ritz-Galerkin](#) techniques; it seeks an approximate solution x_m in the affine subspace $x_0 + \mathcal{K}_m$ by imposing the condition

$$b - A x_m \perp \mathcal{K}_m$$

If $v_1 = \frac{r_0}{\beta}$ with $\beta = \|r_0\|$ in Arnoldi's method. $x_m \in x_0 + \mathcal{K}_m$ means that $\exists y_m \in \mathbb{R}^m$ such that $x_m = x_0 + V_m y_m$.

Exercise 8. How y_m can be computed ?

4.1 The FOM variants

The FOM technique

The full orthogonalization method (FOM) is based on this approach.

FOM ALGORITHM [-10pt]

```

1: Set the initial guess  $x_0 = 0$ 
2:  $r_0 = b$ ;  $\beta = \|r_0\|$   $v_1 = r_0/\|r_0\|$ ;
3: for  $j = 1, 2, \dots$  do
4:    $w_j = Av_j$ 
5:   for  $i = 1$  to  $j$  do
6:      $h_{ij} = v_i^T w_j$ 
7:      $w_j = w_j - h_{i,j}v_i$ 
8:   end for
9:    $h_{j+1,j} = \|w_j\|$ 
10:  If  $h_{j+1,j} = 0$  set  $m = j$  Goto 13
11:   $v_{j+1} = w_j/h_{j+1,j}$ 
12: end for
13:  $y_m = H_m^{-1}(\beta e_1)$  and  $x_m = x_0 + V_m y_m$ .
```

The FOM technique

Proposition 4.3. *The residual vector of the approximate solution x_m computed by FOM is such that*

$$b - Ax_m = -h_{m+1,m}e_m^T y_m v_{m+1}$$

and

$$\|b - Ax_m\| = h_{m+1,m}|e_m^T y_m|. \quad (8)$$

Exercise 9. Give a proof the proposition 4.3.

Cost of FOM

Flops cost:

1. At step j , the Gram-Schmidt process costs: $\approx 4nj$ flops dot product $\approx 2n$, saxpy $\approx 2n$
2. The matrix-vector product costs: $2nnz(A) - n$

Over m steps this leads to $\approx 2 \cdot m \cdot nnz(A) + 2 \cdot m^2 \cdot n$.

Storage: the most consuming part is the basis V_m that is $m \cdot n$.

If the convergence of the algorithm is slow (i.e. m large), it becomes unaffordable.

$\Rightarrow$ Alternative:

1. restart,
2. truncate.

The restarted alternative: FOM(m)

FOM(M) ALGORITHM [-10pt]

```

1: Set the initial guess  $x_0$ 
2:  $r_0 = b - Ax_0$ ;  $\beta = \|r_0\|$   $v_1 = r_0/\|r_0\|$ ;
3: for  $j = 1, \dots, m$  do
4:    $w_j = Av_j$ 
5:   for  $i = 1$  to  $j$  do
6:      $h_{ij} = v_i^T w_j$ 
7:      $w_j = w_j - h_{i,j}v_i$ 
8:   end for
9:    $h_{j+1,j} = \|w_j\|$ 
10:   $v_{j+1} = w_j/h_{j+1,j}$ 
11: end for
12:  $y_m = H_m^{-1}(\beta e_1)$  and  $x_m = x_0 + V_m y_m$ , If converged then Stop.
13: Set  $x_0 = x_m$  goto 2
```

The truncated alternative: DIOM(m)

Governing idea: perform an incomplete orthogonalization in the Arnoldi process (window of width m) and exploit the structure of the resulting H_k to perform an incremental LU factorization of H_k and then derive an update of the iterate at each step of the algorithm.

Advantage: the cost of the orthogonalisation is reduced as well as the cost of the storage (only the last m vectors are stored).

Example with $k = 5$ and $m = 3$

$$H_5 = \begin{pmatrix} h_{11} & h_{12} & h_{13} & & \\ h_{21} & h_{22} & h_{23} & h_{24} & \\ & h_{32} & h_{33} & h_{34} & h_{35} \\ & & h_{43} & h_{44} & h_{45} \\ & & & h_{54} & h_{55} \end{pmatrix}$$

Notice that we still have $AV_k = V_k H_k + h_{k+1,k} v_{k+1} e_k^T$ but V_k is no longer orthogonal.

The basics of the LU factorization

$$A = \begin{pmatrix} a_{1,1} & w^T \\ v & A_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ a_{1,1}^{-1}v & I \end{pmatrix} \begin{pmatrix} a_{1,1} & w^T \\ 0 & A^{(1)} \end{pmatrix}$$

with $A^{(1)} = A_{n-1} - \frac{vw^T}{a_{1,1}}$

Typical Gaussian elimination step k :

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - \frac{a_{ik}^{(k)} \cdot a_{kj}^{(k)}}{a_{kk}^{(k)}}$$

$$H_k = L_k U_k$$

Example with $k = 5$ and $m = 3$

$$H_5 = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ & l_{32} & 1 & & \\ & & l_{43} & 1 & \\ & & & l_{54} & 1 \end{pmatrix} \times \begin{pmatrix} u_{11} & u_{12} & h_{13} & & \\ & u_{22} & u_{23} & h_{24} & \\ & & u_{33} & u_{34} & h_{35} \\ & & & u_{44} & u_{45} \\ & & & & u_{55} \end{pmatrix}$$

$$\equiv \begin{pmatrix} u_{11} & u_{12} & h_{13} & & \\ l_{21} & u_{22} & u_{23} & h_{24} & \\ & l_{32} & u_{33} & u_{34} & h_{35} \\ & & l_{43} & u_{44} & u_{45} \\ & & & l_{54} & u_{55} \end{pmatrix}$$

$$H_6 \equiv \left(\begin{array}{cccc|c} u_{11} & u_{12} & h_{13} & & \\ l_{21} & u_{22} & u_{23} & h_{24} & \\ & l_{32} & u_{33} & u_{34} & h_{35} \\ & & l_{43} & u_{44} & u_{45} \\ & & & l_{54} & u_{55} \\ \hline & & & l_{65} & u_{66} \end{array} \right),$$

where u_{56}, l_{65} and u_{66} (i.e. the update of the LU factorization of H_6) can be computed using

$$\left(\begin{array}{cc|c} u_{44} & u_{45} & h_{46} \\ l_{54} & u_{55} & h_{56} \\ \hline & h_{65} & h_{66} \end{array} \right).$$

The truncated alternative: DIOM(m)

The approximate solution is given by

$$x_k = x_0 + \underbrace{V_k U_k^{-1}}_{P_k} \underbrace{L_k^{-1}(\beta e_1)}_{z_k}$$

then

$$x_k = x_0 + P_k z_k.$$

Because of the structure of U_k , we can easily update P_k and compute its last column using the previous p_i 's and v_k . Equating the last columns of $P_k U_k = V_k$ gives

$$\begin{aligned} \sum_{i=k-m+1}^k u_{ik} p_i &= v_k \\ \Rightarrow p_k &= u_{kk}^{-1} \left(v_k - \sum_{i=k-m+1}^{k-1} u_{ik} p_i \right). \end{aligned}$$

Similarly, we can derive an update for z_k exploiting the structure of L_k (bidiagonal matrix)

$$z_k = \begin{pmatrix} z_{k-1} \\ \xi_k \end{pmatrix}$$

where $\xi_k = -l_{k,k-1} \xi_{k-1}$.

Consequently we have

$$x_k = x_0 + (P_{k-1} p_k) \begin{pmatrix} z_{k-1} \\ \xi_k \end{pmatrix} = \underbrace{x_0 + P_{k-1} z_{k-1}}_{x_{k-1}} + \xi_k p_k.$$

Exercise 10. Show that

$$\|b - Ax_k\| = h_{k+1,k} \left| \frac{\xi_k}{u_{kk}} \right|.$$

DIOM(M) ALGORITHM [-10pt]

- 1: Set the initial guess x_0
- 2: $r_0 = b - Ax_0$; $\beta = \|r_0\|$ $v_1 = r_0/\|r_0\|$;
- 3: **for** $j = 1, \dots$ **do**
- 4: $w_j = Av_j$
- 5: **for** $i = \max\{1, j - m + 1\}$ **to** j **do**
- 6: $h_{i,j} = v_i^T w_j$
- 7: $w_j = w_j - h_{i,j} v_i$
- 8: **end for**
- 9: $h_{j+1,j} = \|w_j\|$ $v_{j+1} = w_j/h_{j+1,j}$
- 10: Update the LU factorization of H_j using the LU factors of H_{j-1}
- 11: $\xi_j = \{\text{if } j = 1 \text{ then } \beta \text{ else } -l_{j,j-1} \xi_{j-1}\}$
- 12: $p_j = u_{jj}^{-1} \left(v_j - \sum_{i=j-k+1}^{j-1} u_{ij} p_i \right)$ (for $i \leq 0$ set $u_{ij} p_i = 0$)
- 13: $x_j = x_{j-1} + \xi_j p_j$
- 14: **end for**

For the sake of simplicity of exposure the DIOM(m) algorithm as been derived using a LU factorization without pivoting, a variant with partial pivoting exists.

4.2 The GMRES variants

The Generalized Minimal RESidual method (GMRES)

The Generalized Minimal RESidual method (GMRES) based on the minimum residual approach. Using the Arnoldi's relation

$$AV_m = V_{m+1}\bar{H}_m,$$

the algorithm builds the iterate $x_m \in x_0 + \mathcal{K}(A, r_0, m)$ such that $\|b - Ax_m\|_2$ is minimal.

Because $x_m \in x_0 + \mathcal{K}(A, r_0, m)$, $\exists y$ such that

$$x_m = x_0 + V_m y.$$

Central equality

Exercise 11. Show that $\|b - Ax_m\|_2 = \|\beta e_1 - \bar{H}_m y\|$

The GMRES method

The GMRES iterate is the vector of $x_0 + \mathcal{K}(A, r_0, m)$ such that

$$\begin{aligned} x_m &= x_0 + V_m y_m \text{ where} \\ y_m &= \arg \min_{y \in \mathbb{R}^m} \|\beta e_1 - \bar{H}_m y\|_2. \end{aligned}$$

The minimizer y_m is inexpensive to compute since it requires only the solution of an $(m+1) \times m$ linear least-squares problem.

The GMRES algorithm

GMRES ALGORITHM [-10pt]

- 1: Set the initial guess x_0 ; $r_0 = b - Ax_0$; $\beta = \|r_0\|$
- 2: $v_1 = r_0/\|r_0\|$;
- 3: **for** $j = 1, \dots, m$ **do**
- 4: $w_j = Av_j$
- 5: **for** $i = 1$ **to** j **do**
- 6: $h_{i,j} = v_i^T w_j$; $w_j = w_j - h_{i,j} v_i$
- 7: **end for**
- 8: $h_{j+1,j} = \|w_j\|$
- 9: If $(h_{j+1,j}) = 0$ goto 12
- 10: $v_{j+1} = w_j/h_{j+1,j}$
- 11: **end for**
- 12: Define the $(j+1) \times j$ upper Hessenberg matrix $\bar{H}_j$
- 13: Solve the least-squares problem $y_j = \arg \min \|\beta e_1 - \bar{H}_j y\|$
- 14: Set $x_j = x_0 + V_j y_j$

[Y. Saad and M. H. Schultz, *GMRES: a generalized minimal residual algorithm for solving nonsymmetric linear systems*, SIAM Journal on Scientific and Statistical Computing, 1986.]

Solution of the least-squares

A stable solution technique to solve $\|\beta e_1 - \bar{H}_m y\|_2$ is to use Givens rotations to transform the Hessenberg matrix into upper triangular form.

$$Q_i = \begin{pmatrix} 1 & & & & & \\ & \dots & & & & \\ & & 1 & & & \\ & & & c_i & s_i & \\ & & & -s_i & c_i & \\ & & & & & 1 & \\ & & & & & & \ddots & \\ & & & & & & & 1 \end{pmatrix} \begin{array}{l} \leftarrow \text{row } i \\ \leftarrow \text{row } i + 1 \end{array}$$

with $c_i^2 + s_i^2 = 1$. Computing $B = Q_i A$ copies all the rows but the i^{th} and $(i+1)^{th}$ from A in B and replaces those two rows by linear combinations of them such that $B_{i+1,i} = 0$ if $s_i = \frac{a_{i+1,i}}{\sqrt{a_{i,i}^2 + a_{i+1,i}^2}}$ and

$$c_i = \frac{a_{i,i}}{\sqrt{a_{i,i}^2 + a_{i+1,i}^2}}.$$

Solving for $\bar{H}_k$

$$\text{Consider } \bar{H}_5 = \begin{pmatrix} h_{11} & h_{12} & h_{13} & h_{14} & h_{15} \\ h_{21} & h_{22} & h_{23} & h_{24} & h_{25} \\ & h_{32} & h_{33} & h_{34} & h_{35} \\ & & h_{43} & h_{44} & h_{45} \\ & & & h_{54} & h_{55} \\ & & & & h_{65} \end{pmatrix} \text{ and } \beta e_1 = \begin{pmatrix} \beta \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

Multiplying by the left by

$$\mathcal{Q}_1 = \begin{pmatrix} c_1 & s_1 & & & \\ -s_1 & c_1 & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & 1 \\ & & & & & 1 \end{pmatrix} \text{ with } \begin{cases} s_1 = \frac{h_{2,1}}{\sqrt{h_{1,1}^2 + h_{2,1}^2}} \\ c_1 = \frac{h_{1,1}}{\sqrt{h_{1,1}^2 + h_{2,1}^2}} \end{cases}.$$

Clearly $\mathcal{Q}_1^T \mathcal{Q}_1 = I$.

$$\bar{H}_5^{(1)} = \begin{pmatrix} h_{11}^{(1)} & h_{12}^{(1)} & h_{13}^{(1)} & h_{14}^{(1)} & h_{15}^{(1)} \\ 0 & h_{22}^{(1)} & h_{23}^{(1)} & h_{24}^{(1)} & h_{25}^{(1)} \\ & h_{32} & h_{33} & h_{34} & h_{35} \\ & & h_{43} & h_{44} & h_{45} \\ & & & h_{54} & h_{55} \\ & & & & h_{65} \end{pmatrix} \text{ and } \bar{g}_1^{(1)} = \begin{pmatrix} c_1 \beta \\ -s_1 \beta \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

We can now multiply by $\mathcal{Q}_2$ to eliminate h_{32} by taking

$$s_2 = \frac{h_{3,2}}{\sqrt{(h_{2,2}^{(1)})^2 + h_{3,2}^2}} \text{ and } c_2 = \frac{h_{2,2}^{(1)}}{\sqrt{(h_{2,2}^{(1)})^2 + h_{3,2}^2}}.$$

This elimination process is continued until the 5th rotation is applied which leads to

$$\bar{H}_5^{(5)} = \begin{pmatrix} h_{11}^{(1)} & h_{12}^{(1)} & h_{13}^{(1)} & h_{14}^{(1)} & h_{15}^{(1)} \\ 0 & h_{22}^{(2)} & h_{23}^{(2)} & h_{24}^{(2)} & h_{25}^{(2)} \\ & 0 & h_{33}^{(3)} & h_{34}^{(3)} & h_{35}^{(3)} \\ & & 0 & h_{44}^{(4)} & h_{45}^{(4)} \\ & & & 0 & h_{55}^{(5)} \\ & & & & 0 \end{pmatrix} \text{ and } \bar{g}^{(5)} = \begin{pmatrix} \gamma_1^{(1)} \\ \gamma_2^{(2)} \\ \gamma_3^{(3)} \\ \gamma_4^{(4)} \\ \gamma_5^{(5)} \\ \gamma_6^{(5)} \end{pmatrix}.$$

Generally, the rotation $\mathcal{Q}_i$ is defined by

$$s_i = \frac{h_{i+1,i}}{\sqrt{(h_{i,i}^{(i-1)})^2 + h_{i+1,i}^2}} \text{ and } c_i = \frac{h_{i,i}^{(i-1)}}{\sqrt{(h_{i,i}^{(i-1)})^2 + h_{i+1,i}^2}}. \quad (9)$$

Define, the unitary matrix

$$Q_m = \mathcal{Q}_m \dots \mathcal{Q}_1 \in \mathbb{R}^{(m+1) \times (m+1)}$$

and let

$$\begin{aligned} \bar{R}_m &= \bar{H}_m^{(m)} = Q_m \bar{H}_m, \\ \bar{g}_m &= \beta Q_m e_1 = (\gamma_1^{(1)} \dots \gamma_m^{(m)} \gamma_{m+1}^{(m)})^T. \end{aligned}$$

Because Q_m is unitary we have

$$\min \|\beta e_1 - \bar{H}_m y\| = \min \|Q_m (\beta e_1 - \bar{H}_m y)\| = \min \|\bar{g}_m - \bar{R}_m y\|$$

The solution of the least-squares problem is obtained by solving the triangular system

$$R_m y = g_m,$$

where $R_m \in \mathbb{R}^{m \times m}$ is obtained from $\bar{R}_m$ by deleting its last row and similarly $g_m \in \mathbb{R}^m$ is obtained from $\bar{g}_m$ by discarding its last component.

The residual of this least-squares problem is the last component of $\bar{g}_m$.

$$R_5 y_5 = g_5$$

$$\bar{R}_5 \begin{pmatrix} h_{11}^{(1)} & h_{12}^{(1)} & h_{13}^{(1)} & h_{14}^{(1)} & h_{15}^{(1)} \\ 0 & h_{22}^{(2)} & h_{23}^{(2)} & h_{24}^{(2)} & h_{25}^{(2)} \\ & 0 & h_{33}^{(3)} & h_{34}^{(3)} & h_{35}^{(3)} \\ & & 0 & h_{44}^{(4)} & h_{45}^{(4)} \\ & & & 0 & h_{55}^{(5)} \\ & & & & 0 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \end{pmatrix} = \bar{g}^{(5)} = \begin{pmatrix} \gamma_1^{(1)} \\ \gamma_2^{(2)} \\ \gamma_3^{(3)} \\ \gamma_4^{(4)} \\ \gamma_5^{(5)} \\ \gamma_6^{(5)} \end{pmatrix}.$$

Some properties of GMRES

Proposition 4.4. 1. The rank of AV_m is equal to the rank of R_m . In particular if $r_{mm} = 0$, then A must be singular.

2. $y_m = \arg \min_{y \in \mathbb{R}^m} \|\beta e_1 - \bar{H}_m y\|$ is given by

$$y_m = R_m^{-1} g_m.$$

3. The residual vector is

$$\begin{aligned} b - Ax_m = V_{m+1}(\beta e_1 - \bar{H}_m y_m) &= V_{m+1} Q_m^T (Q_m \beta e_1 - Q_m \bar{H}_m y_m) \\ &= V_{m+1} Q_m^T (\gamma_{m+1}^{(m)} e_{m+1}) \end{aligned}$$

then

$$\|b - Ax_m\| = |\gamma_{m+1}^{(m)}|.$$

Proof

Exercise 12. Give a proof of the first statement of proposition 4.4

Exercise 13. Give a proof of the last two statements of proposition 4.4

Incremental QR factorization

The QR factorization of $\bar{H}_{m+1}$ can be cheaply computed from the QR factorization of $\bar{H}_m$.

$$\bar{H}_{m+1} = \left(\begin{array}{c|c} \bar{H}_m & \begin{matrix} h_{1,m+1} \\ \vdots \\ h_{m+1,m+1} \end{matrix} \\ \hline 0 & h_{m+2,m+1} \end{array} \right).$$

It is enough to apply the m Givens rotations computed to factor $\bar{H}_m$ to the last column of $\bar{H}_{m+1}$ and build and apply a $(m+1)^{th}$ rotation to zero the $h_{m+2,m+1}$ entry.

$$\bar{H}_6^{(5)} = \left(\begin{array}{ccccc|c} h_{11}^{(1)} & h_{12}^{(1)} & h_{13}^{(1)} & h_{14}^{(1)} & h_{15}^{(1)} & h_{16}^{(1)} \\ 0 & h_{22}^{(2)} & h_{23}^{(2)} & h_{24}^{(2)} & h_{25}^{(2)} & h_{26}^{(2)} \\ & 0 & h_{33}^{(3)} & h_{34}^{(3)} & h_{35}^{(3)} & h_{36}^{(3)} \\ & & 0 & h_{44}^{(4)} & h_{45}^{(4)} & h_{46}^{(4)} \\ & & & 0 & h_{55}^{(5)} & h_{56}^{(5)} \\ & & & & 0 & h_{66}^{(6)} \\ & & & & 0 & h_{76}^{(6)} \end{array} \right) \text{ and } \bar{g}_5^{(6)} = \begin{pmatrix} \gamma_1^{(1)} \\ \gamma_2^{(2)} \\ \gamma_3^{(3)} \\ \gamma_4^{(4)} \\ \gamma_5^{(5)} \\ \gamma_6^{(5)} \\ 0 \end{pmatrix}.$$

We then apply both to $\bar{H}_6^{(5)}$ and $\bar{g}_5^{(6)}$ a 6th Givens rotation defined by

$$s_6 = \frac{h_{76}}{\sqrt{(h_{66}^{(5)})^2 + h_{76}^2}} \text{ and } c_6 = \frac{h_{66}^{(5)}}{\sqrt{(h_{66}^{(5)})^2 + h_{76}^2}}.$$

to get $\bar{R}_6$ and $\bar{g}_6$. In particular we have $\bar{g}_6 = \begin{pmatrix} \gamma_1^{(1)} \\ \gamma_2^{(2)} \\ \gamma_3^{(3)} \\ \gamma_4^{(4)} \\ \gamma_5^{(5)} \\ c_6 \gamma_6^{(5)} \\ -s_6 \gamma_6^{(5)} \end{pmatrix}$ with

$$\|b - Ax_6\| = |\gamma_7^{(6)}| = |-s_6 \gamma_6^{(5)}| = |s_6| |\gamma_6^{(5)}| = |s_6| \|b - Ax_5\|.$$

By induction it can be shown that

$$\|b - Ax_{m+1}\| = |s_m| \|b - Ax_m\|.$$

Remark:

1. The norm of the residual is monotonically decreasing,
2. If $s_m = 0$, the solution is found at step m .

Happy breakdown in GMRES

It exists one possible breakdown in the GMRES algorithm if $h_{k+1,k} = 0$ which prevents to increase the dimension of the search space.

Proposition 4.5. *Let A be a nonsingular matrix. Then the GMRES algorithm breaks down at step k (i.e. $h_{k+1,k} = 0$) iff the iterate x_k is the exact solution of $Ax = b$.*

Proof

Exercise 14. *Show that if there is a breakdown the solution has been found*

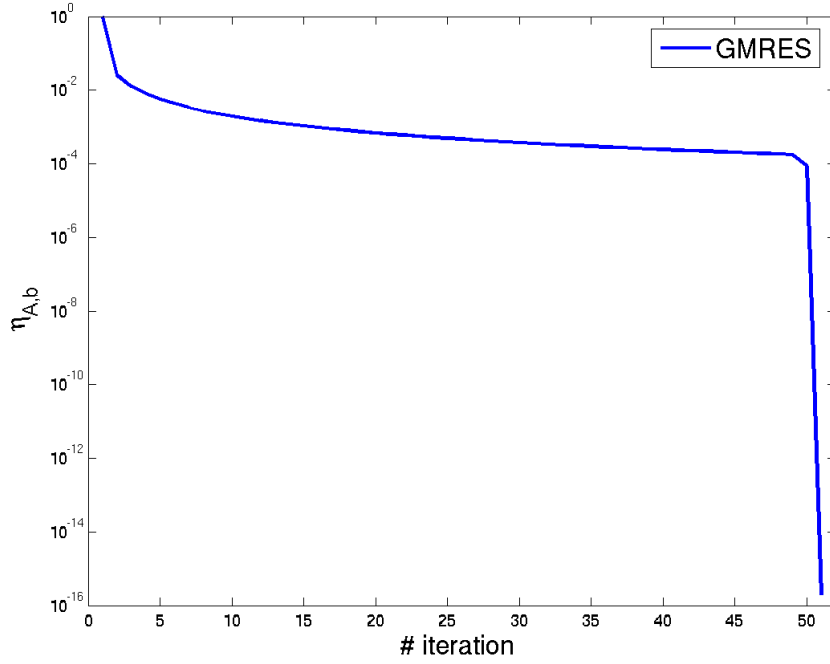
$\Leftarrow$ If x_k is the exact solution of $Ax = b$ then we have $\|b - Ax_k\| = 0 = |\gamma_{k+1}^{(k)}|$. While $\gamma_{k+1}^{(k)} = -s_k \gamma_{k+1}^{(k-1)} = 0$ with $\gamma_{k+1}^{(k-1)} \neq 0$ otherwise convergence would have occur at the previous step (i.e. null residual). From the definition of the Givens rotation

$$s_k = \frac{h_{k+1,k}}{\sqrt{(h_{k,k}^{(k-1)})^2 + h_{k+1,1}^2}} \text{ and } c_k = \frac{h_{k,k}^{(k-1)}}{\sqrt{(h_{k,k}^{(k-1)})^2 + h_{k+1,1}^2}}.$$

we conclude that $s_k = 0 = \frac{h_{k+1,k}}{\sqrt{(h_{k,k}^{(k-1)})^2 + h_{k+1,1}^2}}$.

GMRES: pathological convergence behaviour

The matrix A is a Jordan block of size 50.



Cost of GMRES

- If all but the last Given's rotation implemented by GMRES are applied we end-up with a QR factorization of H_m that can be used by FOM to compute $H_m^{-1}(\beta e_1)$.
- The memory and computational cost of GMRES are very similar to those of FOM.
- To alleviate the cost of the computation (in Gram-Schmidt process) and reduce the memory requirement, the two ideas implemented in FOM(m) and DIOM(m) can be applied to GMRES
 1. restarting GMRES(m),
 2. truncating DQGMRES(m) (Direct Quasi GMRES).

The restarted alternative: GMRES(m)

GMRES(M) ALGORITHM [-10pt]

- 1: Set the initial guess x_0
- 2: $r_0 = b - Ax_0$; $\beta = \|r_0\|$ $v_1 = r_0/\|r_0\|$;
- 3: **for** $j = 1, \dots, m$ **do**
- 4: $w_j = Av_j$
- 5: **for** $i = 1$ **to** j **do**
- 6: $h_{i,j} = v_i^T w_j$; $w_j = w_j - h_{i,j} v_i$
- 7: **end for**
- 8: $h_{j+1,j} = \|w_j\|$; $v_{j+1} = w_j/h_{j+1,j}$
- 9: **end for**
- 10: $y_m = \arg \min \|\beta e_1 - \bar{H}_m y\|$ and $x_m = x_0 + V_m y_m$, If converged then Stop.
- 11: Set $x_0 = x_m$ goto 2

The truncated alternative: DQGMRES(m)

Similar ideas as implemented in DIOM(m), the orthogonality is only maintained among the last m “Arnoldi’s” vectors and an incremental QR factorization is computed that enables the update of the iterate at each step of the algorithm.

Example with $k = 5$ and $m = 2$.

$$\bar{H}_5 = \begin{pmatrix} h_{11} & h_{12} & & & \\ h_{21} & h_{22} & h_{23} & & \\ & h_{32} & h_{33} & h_{34} & \\ & & h_{43} & h_{44} & h_{45} \\ & & & h_{54} & h_{55} \\ & & & & h_{65} \end{pmatrix} \text{ and } g_5 = \begin{pmatrix} \beta \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

Applying the five Givens rotations $\mathcal{Q}_i$ as for regular GMRES only introduce an additional diagonal (each row is a linear combination of that row and the next; this latter introduces the extra term).

$$\bar{R}_5 = \begin{pmatrix} r_{11} & r_{12} & r_{13} & & \\ & r_{22} & r_{23} & r_{24} & \\ & & r_{33} & r_{34} & r_{35} \\ & & & r_{44} & r_{45} \\ & & & & r_{55} \\ & & & & 0 \end{pmatrix} \text{ and } \bar{g}_5 = \begin{pmatrix} \gamma_1^{(1)} \\ \gamma_2^{(2)} \\ \gamma_3^{(3)} \\ \gamma_4^{(4)} \\ \gamma_5^{(5)} \\ \gamma_6^{(5)} \end{pmatrix}.$$

The truncated alternative: DQGMRES(m)

The approximate solution is given by

$$x_k = x_0 + \underbrace{V_k R_k^{-1}}_{P_k} g_k,$$

then

$$x_k = x_0 + P_k g_k.$$

Because of the structure of R_k , we can easily update P_k and compute its last column using the previous p_i 's and v_k . Equating the last columns of $P_k R_k = V_k$ gives

$$\begin{aligned} \sum_{i=k-m+1}^k r_{ik} p_i &= v_k \\ \Rightarrow p_k &= r_{kk}^{-1} \left(v_k - \sum_{i=k-m+1}^{k-1} r_{ik} p_i \right). \end{aligned}$$

Similarly, we can derive an update for g_k which last component is basically updated by the last Givens rotation.

$$\text{We have } \bar{g}_k = \begin{pmatrix} g_{k-1} \\ c_k \gamma_k^{(k-1)} \\ -s_k \gamma_k^{(k-1)} \end{pmatrix} \text{ and } \bar{g}_{k-1} = \begin{pmatrix} \gamma_1^{(1)} \\ \gamma_2^{(2)} \\ \vdots \\ \gamma_k^{(k-1)} \end{pmatrix}$$

$$\text{then } g_k = \begin{pmatrix} g_{k-1} \\ \gamma_k^{(k)} \end{pmatrix}, \text{ with } \gamma_k^{(k)} = c_k \gamma_k^{(k-1)}.$$

Consequently we have

$$x_k = x_0 + (P_{k-1} p_k) \begin{pmatrix} g_{k-1} \\ \gamma_k^{(k)} \end{pmatrix} = \underbrace{x_0 + P_{k-1} g_{k-1}}_{x_{k-1}} + \gamma_k^{(k)} p_k.$$

The residual norm estimate of DQGMRES(m)

We have

$$\begin{aligned}
\|b - Ax_k\| &= \|b - A(x_0 + V_k y_k)\| \\
&= \|r_0 - AV_k y_k\| \\
&= \|V_{k+1}(\beta e_1 - \bar{H}_k y_k)\| \\
&= \|V_{k+1} Q_k^T Q_k (\beta e_1 - \bar{H}_k y_k)\| \\
&= \|V_{k+1} Q_k^T \gamma_{k+1} e_{k+1}\|.
\end{aligned}$$

Let $q = Q_k^T e_{k+1} = (\eta_i)_{i=1, \dots, k+1}$ with $\|q\| = 1$.

$$\begin{aligned}
\|b - Ax_k\| &= |\gamma_{k+1}| \|V_{k+1} q\| \\
&\leq |\gamma_{k+1}| \left(\left\| \sum_{i=1}^{m+1} \eta_i v_i \right\| + \left\| \sum_{i=m+2}^{k+1} \eta_i v_i \right\| \right) \\
&\leq |\gamma_{k+1}| \left(\left(\sum_{i=1}^{m+1} \eta_i^2 \right)^{1/2} + \sum_{i=m+2}^{k+1} |\eta_i| \|v_i\| \right) \\
&\leq |\gamma_{k+1}| \left(\left(\sum_{i=1}^{k+1} \eta_i^2 \right)^{1/2} + \sqrt{k-m} \left(\sum_{i=m+2}^{k+1} \eta_i^2 \right)^{1/2} \right).
\end{aligned}$$

The second term in the last inequality is Cauchy-Schwarz in $\mathbb{R}^{k-m}$. Using again Cauchy-Schwarz in $\mathbb{R}^2$ for

$$1 \cdot a + \sqrt{k-m} \cdot b \leq \sqrt{k-m+1} \sqrt{a^2 + b^2}.$$

Because Q_k is unitary $\|q\| = 1$, then

$$\|b - Ax_k\| \leq \sqrt{k-m+1} |\gamma_{k+1}|.$$

The DQGMRES(m) algorithm

DQGMRES(M) ALGORITHM [-10pt]

- 1: Set the initial guess x_0 $r_0 = b - Ax_0$; $\beta = \|r_0\|$ $v_1 = r_0/\|r_0\|$;
- 2: **for** $j = 1, \dots, k$ **do**
- 3: $w_j = Av_j$
- 4: **for** $i = \max\{1, j-m+1\}$ **to** j **do**
- 5: $h_{i,j} = v_i^T w_j$; $w_j = w_j - h_{i,j} v_i$
- 6: **end for**
- 7: $h_{j+1,j} = \|w_j\|$; $v_{j+1} = w_j/h_{j+1,j}$
- 8: Update the QR factorization of $\bar{H}_j$
- 9: **for** $i = j-k, \dots, j-1$ **do**
- 10: Apply Q_i to the last column of $\bar{H}_j$
- 11: **end for**
- 12: Apply Q_j to $\bar{H}_j$ that is Compute c_j and s_j , $\gamma_{j+1} = -s_j \gamma_j$, $\gamma_j = c_j \gamma_j$, $h_{jj} = c_j h_{j,j} + s_j h_{j+1,j}$
- 13: $p_j = h_{j,j}^{-1} \left(v_j - \sum_{i=k-m}^{j-1} h_{i,j} p_i \right)$; $x_j = x_{j-1} + \gamma_j p_j$
- 14: If converged then stop
- 15: **end for**

[Y. Saad and K. Wu, *DQGMRES: a direct quasi-minimal residual algorithm based on incomplete orthogonalisation*, *Numerical Linear Algebra with Applications*, 1996.]

Some relations between these Krylov methods

Proposition 4.6. 1. *GMRES v.s. FOM* Assume that m steps of GMRES and FOM have been performed (the step of FOM with a singular H_m are skipped). Let $\|r_{m*}^{FOM}\|$ the smallest residual norm achieved in the first m steps and $\|r_m^{GMRES}\|$, the residual norm associated with the iterate computed by GMRES. We have:

$$\|r_m^{GMRES}\| \leq \|r_{m*}^{FOM}\| \leq \sqrt{m} \|r_m^{GMRES}\|.$$

2. *DQGMRES v.s. GMRES* Assume that V_{m+1} the Arnoldi vectors generated by DQGMRES is of full rank. We have the following relation between $\|r_m^{DQGMRES}\|$, the residual norm associated with the iterate computed by DQGMRES at step m and $\|r_m^{GMRES}\|$, the residual norm associated with the iterate computed by GMRES.

$$\|r_m^{DQGMRES}\| \leq \kappa(V_{m+1}) \|r_m^{GMRES}\|$$

Some results on convergence rate

$r_m \in \mathcal{K}(A, r_0, m)$ can be recast in $r_m = q_m(A)r_0$ where $q_m \in \mathbb{P}_m$ with $q_m(0) = 1$ and $\mathbb{P}_m$ is the set of polynomial of degree (m) .

Exercise 15. Show that

$$\|r_m\| \leq \kappa(W) \|r_0\| \min_{q \in \mathbb{P}_m, q(0)=1} \max_{\lambda \in \sigma(A)} |q(\lambda)|.$$

How reads this bound for normal matrices ?

Field of values, numerical radius

The numerical range or field of values of a matrix A is

$$F(A) = \{z^H A z, z \in \mathbb{C}^n, z^H z = 1\}$$

and

$$r(A) = \sup_{z \neq 0} \frac{|z^H A z|}{\|z\|^2} = \sup_{\|z\|=1} |z^H A z|$$

is the numerical radius of A .

The numerical radius has the following properties:

1. $r(A) = 0$ iff $A = 0$,
2. $r(\alpha A) = |\alpha| r(A)$,
3. $\frac{1}{2} \|A\| \leq r(A) \leq \|A\|$,
4. $r(A^m) \leq [r(A)]^m$.

Proof of some properties

1. $r(A) \leq \|A\|$

$$r(A) = \sup_{\|z\|=1} |z^H A z| \leq \sup_{\|z\|=1} (\|z\| \|A z\|) \leq \|A\|.$$

2. $\frac{1}{2} \|A\| \leq r(A)$ $A = \frac{1}{2}(A + A^H) + \frac{1}{2}(A - A^H)$ then

$$\|A\| \leq \frac{1}{2} (\|A + A^H\| + \|A - A^H\|).$$

Furthermore, $A \pm A^H$ is Hermitian then

$$\begin{aligned} \|A \pm A^H\| &= \max_{\|z\|=1} |z^H A z \pm z^H A^H z| \\ &\leq r(A) + r(A^H) = 2r(A) \end{aligned}$$

Proof of $r(A^m) \leq [r(A)]^m$

1. It is enough to show that if $r(A) \leq 1$ then $r(A^m) \leq 1$ (consider $A/r(A)$ if $r(A)$ is not smaller than 1).
2. Let $\{w_k\} = \{e^{2ik\pi/m}\}$ be the set of m th root of unity. We have for $j = 1, \dots, m$ $\{w_k\}_{k=1}^m = \{w_j w_k\}_{k=1}^m$ and $1 - z^m = \prod_{k=1}^m (1 - w_k z)$. Consider the polynomial of degree at most $(m-1)$

$$\begin{aligned} p(z) &= \frac{1}{m} \sum_{j=1}^m \prod_{k=1, k \neq j}^m (1 - w_k z) \\ &= \frac{1}{m} \sum_{j=1}^m \frac{1 - z^m}{1 - w_j z}. \end{aligned}$$

Notice that $p(z) = p(w_1 z) = \dots = p(w_m z)$ for all z , consequently it is a constant polynomial $p(z) = p(0) = 1$.

Proof of $r(A^m) \leq [r(A)]^m$ (cont.)

3. $1 - z^m = \prod_{k=1}^m (1 - w_k z)$ reads $I - A^m = \prod_{k=1}^m (I - w_k A)$, and $1 = \frac{1}{m} \sum_{j=1}^m \prod_{k=1, k \neq j}^m (1 - w_k z)$ reads $I = \frac{1}{m} \sum_{j=1}^m \prod_{k=1, k \neq j}^m (I - w_k A)$.
4. Let x s.t. $\|x\| = 1$,

$$\begin{aligned} 1 - x^H A^m x &= x^H (I - A^m) x = (Ix)^H ((I - A^m)x) \\ &= \left(\frac{1}{m} \sum_{j=1}^m \prod_{k=1, k \neq j}^m (I - w_k A) x \right)^H \left(\prod_{k=1}^m (I - w_k A) x \right) \\ &= \frac{1}{m} \sum_{j=1}^m z_j^H (I - w_j A) z_j \text{ with } z_j = \prod_{k=1, k \neq j}^m (I - w_k A) x \\ &= \frac{1}{m} \sum_{j=1, z_j \neq 0}^m \|z_j\|^2 \left(1 - w_j \left(\frac{z_j}{\|z_j\|} \right)^H A \left(\frac{z_j}{\|z_j\|} \right) \right) \end{aligned}$$

Proof of $r(A^m) \leq [r(A)]^m$ (cont.)

4. If we replace A by $e^{i\theta} A$ in the previous identity

$$1 - e^{im\theta} x^H A^m x = \frac{1}{m} \sum_{j=1, z_j \neq 0}^m \|z_j\|^2 \left(1 - e^{i\theta} w_j \left(\frac{z_j}{\|z_j\|} \right)^H A \left(\frac{z_j}{\|z_j\|} \right) \right).$$

If $r(A) \leq 1$ the real part of the right-hand side is non negative for any θ , so is the real part of the left and side. This is true for any θ then $|x^H A x| \leq 1$ consequently $r(A^m) \leq 1$.

Convergence rate of GMRES

Proposition 4.7. *If the field of value of A is included in a circle of centrum c and radius r such that $r < |c|$ (ie O does not belong to the field of value of A) then*

$$\|b - Ax_k\| \leq 2 \left(\frac{r}{|c|} \right)^k \|b - Ax_0\|.$$

Proof of proposition

Let $q_0(t) = \left(1 - \frac{t}{c}\right)^k$, we have $\|q_0(A)\| \leq 2r \left(q_0(A)\right) = 2r \left((I - A/c)^k\right)$.

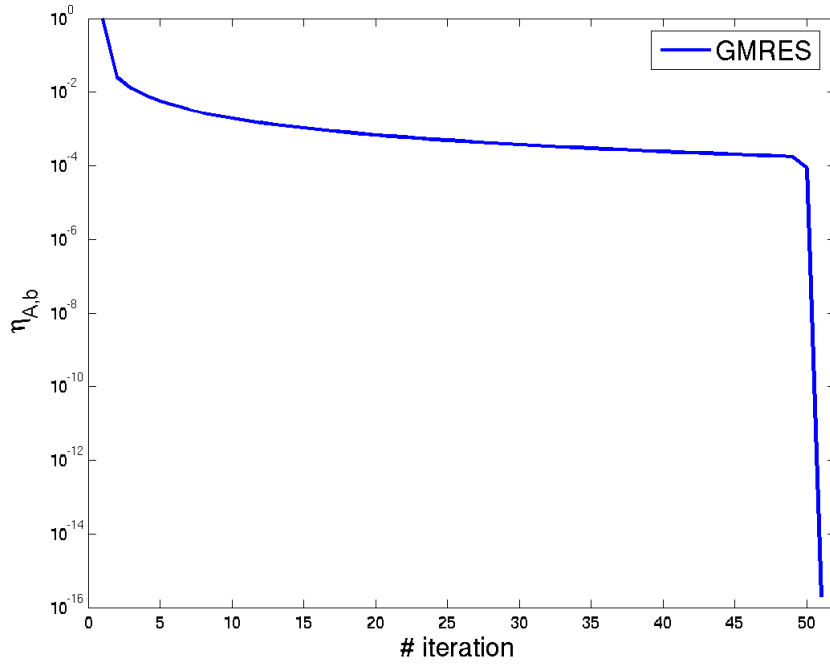
Because $F(A) \subset \mathcal{D}(c, r)$ then $z^H A z = c + r_z e^{i\theta}$ with $r_z \leq r$ for $\|z\| = 1$; consequently $r(I - A/c) \leq r/|c|$. It follows that $r \left((I - A/c)^k\right) \leq \left(\frac{r}{|c|}\right)^k$.

We can conclude that

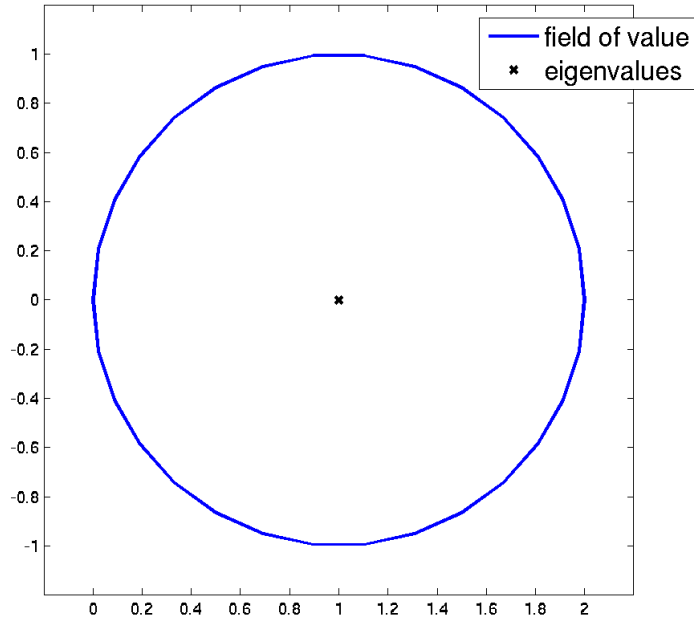
$$\begin{aligned} \|r_k\| &= \min_{q \in \mathbb{P}_m, q(0)=1} \|q(A)r_0\| \\ &\leq \|q_0(A)r_0\| \\ &\leq 2 \left(\frac{r}{|c|}\right)^k \|r_0\| \end{aligned}$$

GMRES: pathological convergence behaviour

The matrix A is a Jordan block of size 50.

**GMRES: pathological convergence behaviour**

The matrix A is a Jordan block of size 50.



On that example: $r=0.998$ and $c=(1.0,0.0)$.

4.3 Strategies for restarted GMRES

Strategies at restart

Motivations

- In GMRES(m), only the approximate solution is kept between cycles/restarts.
- It is often observed that part of the spectrum slow down convergence.
- Attempt to augment the search space with approximated eigenvectors, that are extracted from the Krylov subspace at the end of each cycle.

The Rayleigh-Ritz procedure

For any subspace $S \subseteq \mathbb{C}^n$, a vector $y \in S$ is a Ritz vector of the matrix A with Ritz value λ if

$$w^H(Ay - \lambda y) = 0, \quad \forall w \in S. \quad (10)$$

Let $V \in \mathbb{C}^{n \times m}$ be an orthonormal matrix whose columns span S . Let $z_1 \in \mathbb{C}^m$, $z_2 \in \mathbb{C}^m$ and $y = Vz_1$ and $w = Vz_2$. Equation (10) can be written:

$$z_2^H(V^H AV z_1 - \lambda z_1) = 0, \quad \forall z_2 \in \mathbb{C}^m, \quad (11)$$

which leads to the standard eigenvalue problem:

$$V^H AV g = \lambda g,$$

where z_1 is replaced by g in Equation (11).

The Rayleigh-Ritz procedure

Let (λ_i, g_i) be the eigenpairs of $V^H AV$.

- The λ_i 's are approximate eigenvalues of A and are referred to as Ritz values.
- The Vg_i 's are approximate eigenvectors of A and are referred to as Ritz vectors.

The Rayleigh-Ritz procedure

In the case of Krylov solvers, the choice of $S = \mathcal{K}(A, r_0, m)$ leads to $S = \text{span}\{v_1, \dots, v_m\}$, with $V_m = [v_1, \dots, v_m]$ so that $V_m^H A V_m = H_m$.

Computing the Ritz eigenpairs reduces to solving the standard eigenvalue problem

$$Hg = \lambda g.$$

- Rayleigh-Ritz procedure usually does a better job in finding estimates for exterior eigenvalues,
- A variant of the Rayleigh-Ritz procedure for finding interior eigenvalues should be considered: the harmonic Rayleigh-Ritz procedure.

The harmonic Rayleigh-Ritz procedure

Define the harmonic Ritz values as the Ritz values of A^{-1} with respect to the space AS

$$w^H (A^{-1}y - \alpha y) = 0, \quad \forall w \in AS, \text{ where } \alpha \in \mathbb{C}. \quad (12)$$

Let $V \in \mathbb{C}^{n \times m}$ be an orthonormal matrix whose columns span S . Equation (12) with $y = AVz_1$ and $w = AVz_2$ gives us:

$$z_2^H V^H A^H (A^{-1}AVz_1 - \alpha AVz_1) = z_2^H V^H A^H (Vz_1 - \alpha AVz_1) = 0, \quad \forall z_2$$

which leads to the generalized eigenvalue problem:

$$V^H A^H Vg = \alpha V^H A^H AVg, \quad (13)$$

where z_1 is replaced by g .

The harmonic Rayleigh-Ritz procedure

For $S = \mathcal{K}(A, r_0, m)$, we have $S = \text{span}\{v_1, \dots, v_m\}$ and

- $V_m^H A V_m = H_m$,
- $AV_m = V_{m+1} \bar{H}_m$ with $V_m^H A^H AV_m = \bar{H}_m^H \bar{H}_m = H_m^H H_m + h_{m+1,m}^2 e_m e_m^T$

The generalized eigenvalue problem $\underbrace{V^H A^H V}_{H^H} g = \alpha \underbrace{V^H A^H AV}_{\bar{H}^H \bar{H}} g$ reduces to $(H_m + h_{m+1,m}^2 f e_m^T)g = \alpha^{-1}g$

where $f = H_m^{-H} e_m$.

- The Ritz values α “approximate” the large eigenvalues of A^{-1}
- The value $\theta = \alpha^{-1}$ are the harmonic Ritz values that are expected to approximate the small eigenvalues of A .

Governing ideas

How to extend a Krylov subspace associated with r_0 such that

- at each restart refined information associated with ”smaller” eigenspace can be included,
- an Arnoldi relation of the form $AV_m = V_{m+1} \bar{H}_m$ is satisfied where V_m is still orthonormal.

such that the new algorithm enjoys

- the monotonic residual norm property of GMRES is ensured,
- the main ingredients of the GMRES algorithm are kept.

A first step ahead ..

Let assume that we have performed a first GMRES(m) cycle started with $c = \beta e_1$, we have

- the classical Arnoldi relation: $V_m = V_{m+1}\bar{H}_m$,
- the LS solution: $y_m = \arg \min\{\beta e_1 - \bar{H}_m y\} \in \mathbb{R}^m$,
- the LS residual: $d = c - \bar{H}_m y_m$, so that the new residual is $V_{m+1}d$,
- we assume that the k “smallest” harmonic Ritz vectors $V_m[g_1, \dots, g_k] = V_m G_k = V_{m+1} \begin{bmatrix} g_1 & \cdots & g_k \\ 0 & \cdots & 0 \end{bmatrix} = V_{m+1} G_{k+1}$,

The subspace we would like to restart with is:

$$\text{span} \left(V_{m+1} \begin{bmatrix} g_1 & \cdots & g_k & d \\ 0 & \cdots & 0 & \end{bmatrix} \right)$$

Another step ahead

An orthonormal basis of our new search space can easily be built using the QR factorization:

$$G_{k+1} = \begin{bmatrix} g_1 & \cdots & g_k & d \\ 0 & \cdots & 0 & \end{bmatrix} = P_{k+1} \Gamma_{k+1}$$

then

$$V_{k+1}^{new} = V_{m+1} P_{k+1}$$

is an possible orthonormal basis.

We can notice that $\gamma_{k+1} = \Gamma_{k+1}(:, k+1)$ are the coordinates of the new residual in that basis that correspond to β in classical restarted GMRES (as “ $V_1^{new} = [r_0/\|r_0\|]$ ”).

The last but more tedious step

The matrix

$$\bar{H}_k^{new} = P_{k+1}^H \bar{H}_m P_k$$

is such that

$$A V_k^{new} = V_{k+1}^{new} \bar{H}_k^{new}. \quad (14)$$

From here we can complete our new GMRES cycle using the classical algorithm continuing from v_{k+1}^{new} .

To establish the identity (14) we need a few intermediary results.

A few properties

Proposition 4.8. *The (θ_i, g_i) are such that*

$$\bar{H}_m^H \left(\bar{H}_m g_i - \theta_i \begin{pmatrix} g_i \\ 0 \end{pmatrix} \right) = 0.$$

Proof.

$$V_m^H A^H (A V_m g_i - \theta_i V_m g_i) = 0$$

$$\bar{H}_m^H V_{m+1}^H (V_{m+1} \bar{H}_m g_i - \theta_i V_m g_i) = 0$$

$$\bar{H}_m^H \bar{H}_m g_i - \theta_i \bar{H}_m^H \begin{pmatrix} g_i \\ 0 \end{pmatrix} = 0.$$

□

A few properties

Proposition 4.9. *If $c - \bar{H}_m y_m \neq 0$, $\forall i \in \{1, \dots, k\}$, $\exists \alpha_i \in \mathbb{C}$ such that*

$$\bar{H}_m g_i - \theta_i \begin{pmatrix} g_i \\ 0 \end{pmatrix} = \alpha_i (c - \bar{H}_m y_m).$$

Proof. From the previous property we have that $\bar{H}_m g_i - \theta_i \begin{pmatrix} g_i \\ 0 \end{pmatrix} \in \text{Null}(\bar{H}_m^H)$.

Furthermore, we have $m+1 = \dim(\text{Range}(\bar{H}_m^H)) + \dim(\text{Null}(\bar{H}_m^H))$ because H_m is assumed full rank we have $\dim(\text{Null}(\bar{H}_m^H)) = 1$.

Then $\exists u$, s.t. $\forall i \in \{1, \dots, k\}$ $\bar{H}_m g_i - \theta_i \begin{pmatrix} g_i \\ 0 \end{pmatrix}$ are parallel to u .

$y_m = \arg \min \{c - \bar{H}_m y\}$, then $\bar{H}_m y_m$ is the orthogonal projection of c on $\text{Range}(\bar{H}_m)$ and $(c - \bar{H}_m y_m) \perp \text{Range}(\bar{H}_m)$; that is

$$(c - \bar{H}_m y_m) \in \text{Range}(\bar{H}_m)^\perp = \text{Null}(\bar{H}_m^H),$$

we then have:

$$\forall i \in \{1, \dots, k\} \quad \bar{H}_m g_i - \theta_i \begin{pmatrix} g_i \\ 0 \end{pmatrix} = \alpha_i (c - \bar{H}_m y_m) = \alpha_i d.$$

□

Finally !!

Proposition 4.10. *At restart, we still have the Arnoldi identity*

$$AV_k^{new} = V_{k+1}^{new} \bar{H}_k^{new}.$$

Proof We have, $\forall i \in \{1, \dots, k\}$

$$\begin{aligned} & \bar{H}_m g_i - \theta_i \begin{pmatrix} g_i \\ 0 \end{pmatrix} = \alpha_i d \\ \Rightarrow & V_{m+1} \bar{H}_m g_i - \theta_i V_{m+1} \begin{pmatrix} g_i \\ 0 \end{pmatrix} = \alpha_i V_{m+1} d \\ \Rightarrow & AV_m g_i = V_{m+1} \left(\theta_i \begin{pmatrix} g_i \\ 0 \end{pmatrix} + \alpha_i d \right) \end{aligned}$$

In matrix form this reads

$$AV_m G_k = V_{m+1} G_{k+1} \begin{bmatrix} \theta_1 & 0 & \dots & \dots & 0 \\ 0 & \ddots & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ 0 & \dots & \dots & 0 & \theta_k \\ \alpha_1 & \dots & \dots & \dots & \alpha_k \end{bmatrix} \quad (15)$$

If we denote $G_k = P_k R_k$ the QR factorization of G_k we have the following QR factorization of $G_{k+1} = \begin{bmatrix} G_k & d \end{bmatrix} = P_{k+1} \Gamma_{k+1}$ with $P_{k+1} = \begin{bmatrix} P_k & p_{k+1} \end{bmatrix}$ and $R_{k+1} = \begin{bmatrix} R_k & \gamma_{k+1} \\ 0 & \end{bmatrix}$. Rewriting (15) using these QR factorization we get:

$$\begin{aligned} AV_m P_k R_k &= V_{m+1} P_{k+1} R_{k+1} \begin{bmatrix} \text{diag}\{\theta_1, \dots, \theta_k\} \\ \alpha_1, \dots, \alpha_k \end{bmatrix} \\ \Rightarrow \underbrace{A V_m P_k}_{V_k^{new}} &= \underbrace{V_{m+1} P_{k+1}}_{V_{k+1}^{new}} \underbrace{R_{k+1} \begin{bmatrix} \text{diag}\{\theta_1, \dots, \theta_k\} \\ \alpha_1, \dots, \alpha_k \end{bmatrix}}_{\bar{H}_k^{new} ?} R_k^{-1} \end{aligned}$$

We have, $\forall i \in \{1, \dots, k\}$ $\bar{H}_m g_i - \theta_i \begin{pmatrix} g_i \\ 0 \end{pmatrix} = \alpha_i d$ that reads in matrix form

$$\begin{aligned} \bar{H}_m G_k - \begin{bmatrix} G_k \\ 0 \end{bmatrix} \text{diag}\{\theta_1, \dots, \theta_k\} &= d[\alpha_1, \dots, \alpha_k] \\ \Rightarrow \bar{H}_m P_k R_k &= P_{k+1} R_{k+1} \begin{bmatrix} \text{diag}\{\theta_1, \dots, \theta_k\} \\ \alpha_1, \dots, \alpha_k \end{bmatrix} \\ \Rightarrow P_{k+1}^T \bar{H}_m P_k &= R_{k+1} \begin{bmatrix} \text{diag}\{\theta_1, \dots, \theta_k\} \\ \alpha_1, \dots, \alpha_k \end{bmatrix} R_k^{-1} \end{aligned}$$

GMRES-DR ALGORITHM [-10pt]

- 1: *Initialization.* Choose: m : the maximum size of the subspace where the solution is built, k : the desired number of approximate eigenvectors, tol : the convergence threshold, x_0 : an initial guess. Let $r_0 = b - Ax_0$; $\beta = \|r_0\|_2$, $c = \beta e_1$.
- 2: *First iteration.* Apply standard GMRES(m). It generates V_{m+1} , $\bar{H}_m$ and the new approximate solution $x_m = x_0 + V_m y_m$ where $y_m = \text{argmin}_y \|c - \bar{H}_m y\|_2$. Let $x_0 = x_m$, $r_0 = b - Ax_m$.
- 3: *Compute Harmonic Ritz vectors.* Compute the k smallest eigenpairs (θ_i, g_i) of the matrix: $H_m + h_{m+1,m}^2 \bar{H}_m^{-H} e_m e_m^T$. Then store the g_i , $i = 1 \dots k$, into the matrix G_k .
- 4: *Append the quasi-residual $\beta e_1 - \bar{H}_m y_m$ to G_k .* Then add a zero row to G_k and the column $c - \bar{H}_m y_m$. In the two cases, the new matrix is noted G_{k+1} and its dimensions are $(m+1) \times (k+1)$.
- 5: *Orthonormalization of the columns of G_{k+1} .* Make a QR -factorisation of G_{k+1} : $G_{k+1} = P_{k+1} \Gamma_{k+1}$, store P_{k+1} and the last column of Γ_{k+1} , γ_{k+1} .
- 6: *Update V_{k+1} , V_k and $\bar{H}_k$.* Let $V_{k+1}^{new} = V_{m+1} P_{k+1}$, and $\bar{H}_k^{new} = P_{k+1}^H \bar{H}_m P_k$. Then let $V_{k+1} = V_{k+1}^{new}$, and $\bar{H}_k = \bar{H}_k^{new}$.
- 7: *Arnoldi iteration.* Form the rest of V_{m+1} and $\bar{H}_m$ applying the Arnoldi iteration starting with v_{k+1} .
- 8: *Form the approximate solution.* Let $c = \begin{bmatrix} I_{k+1} \\ 0 \end{bmatrix} \gamma_{k+1}$ and solve $\min \|c - \bar{H}_m y\|$ for y_m . Compute the approximate solution $x_m = x_0 + V_m y_m$ and the residual $r = V_{m+1}^H (c - \bar{H}_m y)$.
- 9: *Check convergence.*
- 10: *If convergence not achieved restart.* Let $x_0 = x_m$ and $r_0 = r$. Go to 3.

[R. B. Morgan, *GMRES with deflated restarting*, SIAM J. Scientific Computing, 2002.]

Some computational concerns

In real arithmetic

- the real basis associated with conjugate eigenvalues should be considered for forming G_k ;
- the number of recycled vectors should be adapted if the complete real basis cannot fit into the k vectors.

5 Symmetric Krylov solvers based on the Lanczos approach

Arnoldi for symmetric problems: the Lanczos method

Proposition 5.11. Assume that the Arnoldi's method is applied to a real symmetric matrix A . The matrix H_m is tridiagonal and symmetric.

The standard notation used to described the Lanczos algorithm is to used T_m rather than H_m and $\alpha_j \equiv h_{j,j}$ and $\beta_j \equiv h_{j-1,j}$.

$$T_m = \begin{pmatrix} \alpha_1 & \beta_2 & & & \\ \beta_2 & \alpha_2 & \beta_3 & & \\ & \ddots & \ddots & \ddots & \\ & & \beta_{m-1} & \alpha_{m-1} & \beta_m \\ & & & \beta_m & \alpha_m \end{pmatrix}.$$

Lanczos



Cornelius Lanczos

1893-1974: Hungary.

Lanczos worked on relativity and mathematical physics and invented what is now called the Fast Fourier Transform. He worked with Einstein in Berlin, for the Boeing Aircraft Company, at Purdue University and the Department at the Dublin Institute for Advance Study in Ireland.

The Lanczos algorithm

LANCZOS ALGORITHM [-10pt]

- 1: Choose an initial vector v_1 of norm equal to one.
- 2: $\beta_1 = 0$ and $v_0 = 0$
- 3: **for** $j = 1, 2, \dots, m$ **do**
- 4: $w_j = Av_j - \beta_j v_{j-1}$
- 5: $\alpha_j = w_j^T v_j$
- 6: $w_j = w_j - \alpha_j v_j$
- 7: $\beta_{j+1} = \|w_j\|$. If $\beta_{j+1} = 0$ then Stop.
- 8: $v_{j+1} = w_j / \beta_{j+1}$
- 9: **end for**

The following “Arnoldi” relations read:

$$\begin{aligned} AV_m &= V_m T_m + \beta_{m+1} v_{m+1} e_m^T, \\ V_m^T AV_m &= T_m. \end{aligned}$$

Notice that only 3 vectors need to be stored to build the orthogonal basis.

5.1 Lanczos algorithm for linear systems

Lanczos algorithm for linear systems

Similarly to the FOM approach, this technique belongs to the Ritz-Galerkin techniques; it seeks an approximate solution x_m in the affine subspace $x_0 + \mathcal{K}_m$ by imposing the condition

$$b - Ax_m \perp \mathcal{K}_m.$$

Exercise 16. How can be expressed x_m .

The Lanczos algorithm

LANCZOS FOR LINEAR SYSTEMS [-10pt]

- 1: Compute $r_0 = b - Ax_0$, $\beta = \|r_0\|$ and $v_1 = r_0 / \beta$.
- 2: **for** $j = 1, 2, \dots, m$ **do**
- 3: $w_j = Av_j - \beta_j v_{j-1}$ (If $j = 1$ set $\beta_1 v_0 = 0$)
- 4: $\alpha_j = w_j^T v_j$
- 5: $w_j = w_j - \alpha_j v_j$
- 6: $\beta_{j+1} = \|w_j\|$. If $\beta_{j+1} = 0$ then set $m = j$ goto 9
- 7: $v_{j+1} = w_j / \beta_{j+1}$
- 8: **end for**
- 9: Set $T_m = \text{tridiag}(\beta_i, \alpha_i, \beta_{i+1})$ and $V_m = (v_1, \dots, v_m)$.
- 10: Compute $y_m = T_m^{-1}(\beta e_1)$ and $x_m = x_0 + V_m y_m$.

Residual property

Exercise 17. Show that the residual associated with x_m is colinear to v_{m+1} ?

The D-Lanczos variant

Because T_m is tridiagonal, we can compute an incremental LU factorization so that

1. the iterate can be updated at each step of the Lanczos method,
2. we do not have to store all the basis vectors to compute x_m .

The technique is very similar to the one implemented by truncated variants of FOM and GMRES, namely DIOM(m) and DQGMRES(m).

$$\begin{aligned}
 T_5 &= \begin{pmatrix} \alpha_1 & \beta_2 & & & \\ \beta_2 & \alpha_2 & \beta_3 & & \\ & \beta_3 & \alpha_3 & \beta_4 & \\ & & \beta_4 & \alpha_4 & \beta_5 \\ & & & \beta_5 & \alpha_5 \end{pmatrix} \\
 &= \begin{pmatrix} 1 & & & & \\ \lambda_2 & 1 & & & \\ & \lambda_3 & 1 & & \\ & & \lambda_4 & 1 & \\ & & & \lambda_5 & 1 \end{pmatrix} \begin{pmatrix} \eta_1 & \beta_2 & & & \\ & \eta_2 & \beta_3 & & \\ & & \eta_3 & \beta_4 & \\ & & & \eta_4 & \beta_5 \\ & & & & \eta_5 \end{pmatrix}.
 \end{aligned}$$

The approximate solution is given by

$$x_m = x_0 + \underbrace{V_m U_m^{-1}}_{P_m} \underbrace{L_m^{-1}(\beta e_1)}_{z_m},$$

then

$$x_m = x_0 + P_m z_m.$$

Because of the structure of U_m , we can easily update P_m and compute its last column using the previous p_i 's and v_m . Equating the last columns of $P_m U_m = V_m$ gives

$$\begin{aligned}
 \eta_m p_m + \beta_m p_{m-1} &= v_m \\
 \Rightarrow p_m &= \eta_m^{-1} (v_m - \beta_m p_{m-1}).
 \end{aligned}$$

Notice that β_i in the U_m factor are the β_i computed by the Lanczos algorithm; they are unchanged by the Gaussian elimination. The η_m and λ_m results from the Gaussian elimination applied to a tridiagonal matrix. We have:

$$\begin{aligned}
 \lambda_m &= \frac{\beta_m}{\eta_{m-1}}, \\
 \eta_m &= \alpha_m - \lambda_m \beta_m.
 \end{aligned}$$

Similarly, we can derive an update for z_m exploiting the structure of L_m (bidiagonal matrix)

$$z_m = \begin{pmatrix} z_{m-1} \\ \xi_m \end{pmatrix},$$

where $\xi_m = -\lambda_m \xi_{m-1}$.

Consequently we have

$$x_m = x_0 + (P_{m-1} p_m) \begin{pmatrix} z_{m-1} \\ \xi_m \end{pmatrix} = \underbrace{x_0 + P_{m-1} z_{m-1}}_{x_{m-1}} + \xi_m p_m$$

and

$$\|r_m\| = \beta_{m+1} \left| \frac{\xi_m}{\eta_m} \right|.$$

D-LANCZOS ALGORITHM [-10pt]

```

1: Set  $r_0 = b - Ax_0$ ;  $\xi_1 = \beta = \|r_0\|$  and  $v_1 = r_0/\beta$ 
2: Set  $\lambda_1 = \beta_1 = 0$ ,  $p_0 = 0$ 
3: for  $j = 1, \dots$  do
4:   Compute  $w_j = Av_j - \beta_j v_{j-1}$  and  $\alpha_j = w_j^T v_j$ 
5:   If  $j > 1$  the computed  $\lambda_j = \frac{\beta_j}{\eta_{j-1}}$  and  $\xi_j = -\lambda_j \xi_{j-1}$ 
6:    $\eta_j = \alpha_j - \lambda_j \beta_j$ 
7:    $p_j = \eta_j^{-1} (v_j - \beta_j p_{j-1})$ 
8:    $x_j = x_{j-1} + \xi_j p_j$ 
9:   If converged the Stop
10:   $w_j = w_j - \alpha_j v_j$ 
11:   $\beta_{j+1} = \|w_j\|$ ,  $v_{j+1} = w_j/\beta_{j+1}$ 
12: end for

```

Some important properties

Proposition 5.12. *Let $r_m = b - Ax_m$ and p_m the vectors generated by the D-Lanczos algorithm. We have:*

1. *the residual vector r_m is colinear to v_{m+1} . As a consequence, the residual vectors are orthogonal to each other.*
2. *the vectors p_i are A -conjugate, that is: $\forall i \neq j, p_i^T A p_j = 0$.*

Proof

1. We have already seen that

$$\begin{aligned}
b - Ax_m &= b - A(x_0 + V_m y_m) \\
&= r_0 - AV_m y_m \\
&= r_0 - AV_m y_m \\
&= \underbrace{V_m(\beta e_1 - T_m y_m)}_O - \beta_{m+1} v_{m+1} e_m^T y_m.
\end{aligned}$$

2. We will show that $P_m^T A P_m$ is a diagonal matrix, where $P_m = V_m U_m^{-1}$.

$$\begin{aligned}
P_m^T A P_m &= U_m^{-T} V_m^T A V_m U_m^{-1} \\
&= U_m^{-T} T_m U_m^{-1} \\
&= U_m^{-T} L_m.
\end{aligned}$$

The product of two lower triangular matrices is a lower triangular matrix, then $U_m^{-T} L_m$ is lower triangular, because $U_m^{-T} L_m = P_m^T A P_m$ it is also symmetric; consequently it is diagonal.

5.2 The conjugate gradient algorithm

Derivation of the conjugate gradient algorithm

The above properties can be used to derived a variant of D-Lanczos by imposing the orthogonality and conjugacy conditions. To end-up with the most classical form of CG, the indexing of the p_i 's will start at zero (in particular $p_0 = r_0$) and we use the classical notation. As in D-Lanczos, we will have $x_{j+1} = x_j + \alpha_j p_j$, consequently $r_{j+1} = r_j - \alpha_j A p_j$.

Because of the orthogonality of the r_i we have

$$\alpha_j = \frac{r_j^T r_j}{r_j^T A p_j}.$$

We also have (with the change of indexing on p_i 's and an appropriate scaling of p_i 's)

$$p_{j+1} = r_{j+1} + \beta_j p_j,$$

then

$$r_j^T A p_j = (p_j - \beta_{j-1} p_{j-1})^T A p_j = p_j^T A p_j$$

and

$$\alpha_j = \frac{r_j^T r_j}{p_j^T A p_j}.$$

We now write that $p_{j+1} = r_{j+1} + \beta_j p_j$ is orthogonal to $A p_j$ to get

$$\beta_j = -\frac{r_{j+1}^T A p_j}{p_j^T A p_j}.$$

From the residual relation

$$A p_j = \alpha_j^{-1} (r_{j+1} - r_j),$$

and from the definition of $\alpha_j = \frac{r_j^T r_j}{p_j^T A p_j}$ we have

$$A p_j = \frac{p_j^T A p_j}{r_j^T r_j} (r_{j+1} - r_j)$$

that plugs back in the definition of β_j reads

$$\beta_j = \frac{r_{j+1}^T r_{j+1}}{r_j^T r_j}.$$

The conjugate gradient algorithm

Putting together all the above relations gives the following algorithm

CG ALGORITHM [-10pt]

- 1: $r_0 = b - A x_0, p_0 = r_0.$
- 2: **for** $j = 0, 1, \dots$ **do**
- 3: $\alpha_j = (r_j^T r_j) / (p_j^T A p_j)$
- 4: $x_{j+1} = x_j + \alpha_j p_j$
- 5: $r_{j+1} = r_j - \alpha_j A p_j$
- 6: $\beta_j = (r_{j+1}^T r_{j+1}) / (r_j^T r_j)$
- 7: $p_{j+1} = r_{j+1} + \beta_j p_j$
- 8: **end for**

Notice that the α_j and β_j are different from those of D-Lanczos. In term of storage CG requires four vectors $(x, p, A p, r)$ versus fives for D-Lanczos (v_m, v_{m-1}, w, p, x) .

[M .R. Hestenes and E. Stiefel, *Methods of conjugate gradients for solving linear systems*, J. Res. Nat. Bur. Standards, vol. 49, 409–435, 1952.]

Some important remarks

The algorithm relies on a LU factorization without pivoting of a symmetric tridiagonal matrix. This factorization might not exist and CG can break-down.

1. For symmetric positive definite (SPD) matrices, T_m is always non singular and a stable LU decomposition exists. CG is then the method of choice for this class of matrices.
2. For symmetric indefinite matrices a technique based on a LQ decomposition of T_m exists that is known as SYMMLQ.
3. For symmetric indefinite matrices, the solution of the linear system involving T_m can be replaced by the solution of a linear least-square involving T_m . This method is known as MINRES and belong to the minimum norm error class of Krylov solvers.

Property of CG in the SPD case

Proposition 5.13. *Because A is SPD, the bilinear form $x^T A y$ define an inner product. The Ritz-Galerkin condition $b - Ax_m \perp \mathcal{K}_m$ can be written $A(x_m - x^*) \perp \mathcal{K}_m$ which also reads $(x_m - x^*) \perp_A \mathcal{K}_m$. This latter condition implies that*

$$\|x_m - x^*\|_A$$

is minimal over $\mathcal{K}_m$.

Proof. Let $y \in \mathcal{K}_m$ with $y \neq x_m$

$$\begin{aligned} \|y - x^*\|_A^2 &= \|(y - x_m) + (x_m - x^*)\|_A^2 \\ &= \|y - x_m\|_A^2 + 2 \underbrace{(y - x_m)^T A (x_m - x^*)}_{0 \text{ because } (y - x_m) \in \mathcal{K}_m} + \|x_m - x^*\|_A^2 \\ &= \|y - x_m\|_A^2 + \|x_m - x^*\|_A^2 \\ &> \|x_m - x^*\|_A^2 \end{aligned}$$

□

Bound on the convergence rate of CG

We have the following upper-bound for the convergence rate of CG in the SPD case.

Proposition 5.14. *Let x_m be the m^{th} iterate generated by the CG algorithm then*

$$\|x_m - x^*\|_A \leq 2 \cdot \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^m \|x_0 - x^*\|_A.$$

Proof

$x_m = x_0 + \mathcal{K}(A, r_0, m)$ can be recast in $x_m = x_0 + q_m(A)r_0$ where $q_m \in \mathbb{P}_{m-1}$ and $\mathbb{P}_{m-1}$ is the set of polynomial of degree $(m-1)$. We then have

Exercise 18. *Show that*

$$\|x_m - x^*\|_A = \min_{p \in \mathbb{P}_m, p(0)=1} \|p(A)(x_0 - x^*)\|_A.$$

Let $\lambda_i, i = 1, \dots, n$ be the eigenvalues of A and $\xi = (\xi_1, \dots, \xi_n)^T$ where ξ_i for $i = 1, \dots, n$ are the components of $(x_0 - x^*)$ in this eigenbasis V . We have $A = V\Lambda V^T$, and $(x_m - x^*) = V\xi$ then

$$\begin{aligned} p(A)(x_0 - x^*) &= Vp(\Lambda)V^T(V\xi) \\ &= Vp(\Lambda)\xi. \end{aligned}$$

$$\begin{aligned} \|p(A)(x_0 - x^*)\|_A^2 &= (Vp(\Lambda)\xi)^T A (Vp(\Lambda)\xi) \\ &= (p(\Lambda)\xi)^T V^T (V\Lambda V^T) (Vp(\Lambda)\xi) \\ &= (p(\Lambda)\xi)^T \Lambda (p(\Lambda)\xi) \\ &= \xi^T p(\Lambda)^T ((\Lambda p(\Lambda))\xi) = \xi^T p(\Lambda)^T p(\Lambda) (\Lambda \xi) \\ &= \sum_{i=1}^n p(\lambda_i)^2 \lambda_i \xi_i^2 \\ &\leq \max_i (p(\lambda_i)^2) \sum_{i=1}^n \lambda_i \xi_i^2 \\ &\leq \max_i (p(\lambda_i)^2) (\Lambda \xi)^T \xi \text{ with } \xi = V^T(x_m - x^*) \\ &\leq \max_i (p(\lambda_i)^2) \|x_0 - x^*\|_A^2 \\ &\leq \max_{\lambda \in [\lambda_{\min}, \lambda_{\max}]} (p(\lambda))^2 \|x_0 - x^*\|_A^2. \end{aligned}$$

This shows that

$$\|x_m - x^*\|_A \leq \min_{p \in \mathbb{P}_m, p(0)=1} \left(\max_{\lambda \in [\lambda_{\min}, \lambda_{\max}]} |p(\lambda)| \right) \|x_0 - x^*\|_A. \quad (16)$$

Using an approximation theory result we have

$$\min_{p \in \mathbb{P}_m, p(0)=1} \left(\max_{\lambda \in [\lambda_{\min}, \lambda_{\max}]} |p(\lambda)| \right) \leq \frac{1}{|C_m(\frac{\lambda_{\max} + \lambda_{\min}}{\lambda_{\max} - \lambda_{\min}})|} \quad (17)$$

where $C_m(t)$ is the Chebyshev polynomial of the first kind of degree m . For $|t| > 1$ we have

$$C_m(t) = \frac{1}{2} \left[\left(t + \sqrt{t^2 - 1} \right)^m + \left(t - \sqrt{t^2 - 1} \right)^{-m} \right] \geq \frac{1}{2} \left(t + \sqrt{t^2 - 1} \right)^m.$$

Let $\eta = \frac{\lambda_{\min}}{\lambda_{\max} - \lambda_{\min}}$ ($\eta + 1 = \frac{\lambda_{\max}}{\lambda_{\max} - \lambda_{\min}}$) we have $\frac{\lambda_{\max} + \lambda_{\min}}{\lambda_{\max} - \lambda_{\min}} = 1 + 2\eta$

$$\begin{aligned} C_m \left(\frac{\lambda_{\max} + \lambda_{\min}}{\lambda_{\max} - \lambda_{\min}} \right) &= C_m(1 + 2\eta) \\ &\geq \frac{1}{2} \left(1 + 2\eta + \sqrt{(1 + 2\eta)^2 - 1} \right)^m \\ &\geq \frac{1}{2} \left(1 + 2\eta + 2\sqrt{\eta(\eta + 1)} \right)^m \\ &\geq \frac{1}{2} \left(\left(\sqrt{\eta} + \sqrt{\eta + 1} \right)^2 \right)^m \\ &\geq \frac{1}{2} \left(\frac{(\sqrt{\lambda_{\min}} + \sqrt{\lambda_{\max}})^2}{\lambda_{\max} - \lambda_{\min}} \right)^m \\ &\geq \frac{1}{2} \left(\frac{\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}}}{\sqrt{\lambda_{\max}} - \sqrt{\lambda_{\min}}} \right)^m \\ &\geq \frac{1}{2} \left(\frac{\sqrt{\kappa} + 1}{\sqrt{\kappa} - 1} \right)^m. \end{aligned}$$

This implies

$$\frac{1}{|C_m(\frac{\lambda_{\max} + \lambda_{\min}}{\lambda_{\max} - \lambda_{\min}})|} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^m.$$

Plugging this bound in (17) and then in (16) completes the proof.

Estimate of the A-norm of the error in CG

Proposition 5.15. *The CG iterates are such that*

$$\|x - x_m\|_A^2 = \sum_{i=m}^{m+d-1} \alpha_i \|r_i\|_2^2 + \|x - x_{m+d}\|_A^2.$$

The A-norm of the error is decreasing, if d is such that

$$\|x - x_{m+d}\|_A^2 \ll \|x - x_m\|_A^2$$

then

$$\|x - x_m\|_A^2 \approx \sum_{i=m}^{m+d-1} \alpha_i \|r_i\|_2^2.$$

Proof

We first need to show that: $\forall m > 0, r_{m+1}^T p_m = 0$.

This can be done by induction:

1. $m = 1$ $p_0 = r_0$, then $r_1^T p_0 = r_1^T r_0 = 0$ because the r_m are orthogonal.
2. $m > 1$

$$\begin{aligned} r_{m+1}^T p_m &= (r_m - \alpha_m A p_m)^T p_m \\ &= r_m^T p_m - \alpha_m p_m^T A p_m \text{ with } \alpha_m p_m^T A p_m = \|r_m\|^2 \\ &= r_m^T (r_m + \beta_{m-1} p_{m-1}) - \|r_m\|^2 \\ &= \|r_m\|^2 + \beta_{m-1} r_m^T p_{m-1} - \|r_m\|^2 \\ &= \beta_{m-1} r_m^T p_{m-1} = 0. \end{aligned}$$

$$\begin{aligned}
\|x - x_m\|_A^2 - \|x - x_{m+1}\|_A^2 &= \|x - x_{m+1} - (x_{m+1} - x_m)\|_A^2 - \|x - x_{m+1}\|_A^2 \\
&= \|x - x_{m+1}\|_A^2 - 2(x - x_{m+1})^T A(x_{m+1} - x_m) \\
&\quad + \|x_{m+1} - x_m\|_A^2 - \|x - x_{m+1}\|_A^2 \\
&= \|x_{m+1} - x_m\|_A^2 - 2r_{m+1}^T \underbrace{(x_{m+1} - x_m)}_{\alpha_m p_m} \\
&= (\alpha_m p_m)^T \underbrace{A(\alpha_m p_m)}_{r_m - r_{m+1}} - 2\alpha_m r_{m+1}^T p_m \\
&= \alpha_m p_m^T r_m \\
&= \alpha_m (r_m + \beta_{m-1} p_{m-1})^T r_m \\
&= \alpha_m \|r_m\|^2.
\end{aligned}$$

Then

$$\|x - x_m\|_A^2 = \|x - x_{m+1}\|_A^2 + \alpha_m \|r_m\|^2,$$

that can be easily extended

$$\|x - x_m\|_A^2 = \|x - x_{m+d}\|_A^2 + \sum_{i=m}^{m+d-1} \alpha_i \|r_i\|^2.$$

6 The Lanczos biorthogonalization approach

Lanczos biorthogonalization

Constructing an orthogonal basis of a Krylov space when A is unsymmetric is expensive both in memory and in floating point iterations (Gram-Schmidt procedure).

We now consider methods that are based on the biorthogonalization algorithm due to Lanczos. This algorithm builds a pair of biorthogonal bases for the two subspaces

$$\mathcal{K}(A, m, v_1) = \text{span}\{v_1, Av_1, \dots, A^{m-1}v_1\}$$

and

$$\mathcal{K}(A^T, m, w_1) = \text{span}\{w_1, A^T w_1, \dots, (A^T)^{m-1}w_1\}.$$

The Lanczos biorthogonalization algorithm

LANCZOS BIORTHOGONALIZATION ALGORITHM [-10pt]

- 1: Choose two vectors v_1 and w_1 such that $v_1^T w_1 = 1$
- 2: Set $\beta_1 = \delta_1 = 0$, $w_0 = v_0 = 0$
- 3: **for** $j = 1, \dots, m$ **do**
- 4: $\alpha_j = w_j^T A v_j$
- 5: $\hat{v}_{j+1} = A v_j - \alpha_j v_j - \beta_j v_{j-1}$
- 6: $\hat{w}_{j+1} = A^T w_j - \alpha_j w_j - \delta_j w_{j-1}$
- 7: $\delta_{j+1} = \sqrt{|\hat{v}_{j+1}^T \hat{w}_{j+1}|}$; If $\delta_{j+1} = 0$ then Stop
- 8: $\beta_{j+1} = \hat{v}_{j+1}^T \hat{w}_{j+1} / \delta_{j+1}$
- 9: $w_{j+1} = \hat{w}_{j+1} / \beta_{j+1}$
- 10: $v_{j+1} = \hat{v}_{j+1} / \delta_{j+1}$
- 11: **end for**

There are many possible choices for δ_{j+1} and β_{j+1} , the only constraint is that $\delta_{j+1}\beta_{j+1} = \hat{v}_{j+1}^T \hat{w}_{j+1}$.

Notice from the algorithm that the vectors v_i belong to $\mathcal{K}(A, m, v_1)$ and w_i belong to $\mathcal{K}(A^T, m, w_1)$.

Possible break-downs and cure in the unsymmetric Lanczos

The Lanczos biorthogonalization algorithm will abort if $\hat{v}_{j+1}^T \hat{w}_{j+1} = 0$. This can happen in different situations:

1. $\hat{v}_{j+1} = 0$ Happy break-down, an invariant subspace has been build and the approximate solution built in that space is exact.
2. $\hat{w}_{j+1} = 0$ The remedy is to restart the algorithm.
3. $\hat{v}_{j+1} \neq 0$ and $\hat{w}_{j+1} \neq 0$ but $\hat{v}_{j+1}^T \hat{w}_{j+1} = 0$ This situation is often refer to *serious break down*. Possible remedies are to restart or the look-ahead variants. These latter define pairs v_{j+2}, w_{j+2} even though v_{j+1}, w_{j+1} are undefined. These look-ahead technique are fairly complex and the tridiagonal structure is lost. Therefore, the simpler remedy that is restarting the Lanczos procedure is often preferred.

Let

$$T_m = \begin{pmatrix} \alpha_1 & \beta_2 & & & \\ \delta_2 & \alpha_2 & \beta_3 & & \\ & \ddots & \ddots & \ddots & \\ & & \delta_{m-1} & \alpha_{m-1} & \beta_m \\ & & & \delta_m & \alpha_m \end{pmatrix}.$$

Proposition 6.16. *If the algorithm does not break down before step m , the vectors $V = (v_1, \dots, v_m)$ and $W = (w_1, \dots, w_m)$ form a biorthogonal system; that is:*

$$V_m^T W_m = I_m.$$

Moreover, V_m is a basis of $\mathcal{K}(A, m, v_1)$ and W_m a basis of $\mathcal{K}(A^T, m, w_1)$ and the following relations hold

$$AV_m = V_m T_m + \delta_{m+1} v_{m+1} e_m^T, \quad (18)$$

$$A^T W_m = W_m T_m^T + \beta_{m+1} w_{m+1} e_m^T, \quad (19)$$

$$W_m^T AV_m = T_m. \quad (20)$$

Proof

We first show by induction that $V_m^T W = I_m$. We assume that is is true for m and show it for $m+1$. This is clearly true for $m=1$, as $v_1^T w_1 = 1$

Fist we show that $v_{m+1}^T w_i = 0$ for $i = 1, \dots, m$. For $i = m$ [-10pt]

$$\langle v_{m+1}, w_m \rangle = \delta_{m+1}^{-1} (\underbrace{\langle Av_m, w_m \rangle}_{\alpha_m} - \underbrace{\alpha_m \langle v_m, w_m \rangle}_1 - \underbrace{\beta_m \langle v_{m-1}, w_m \rangle}_0).$$

For $i < m$ [-10pt]

$$\begin{aligned} \langle v_{m+1}, w_i \rangle &= \delta_{m+1}^{-1} (\langle Av_m, w_i \rangle - \alpha_m \langle v_m, w_i \rangle - \beta_m \langle v_{m-1}, w_i \rangle) \\ &= \delta_{m+1}^{-1} (\langle v_m, A^T w_i \rangle - \beta_m \langle v_{m-1}, w_i \rangle) \\ &= \delta_{m+1}^{-1} (\langle v_m, \beta_{i+1} w_{i+1} + \alpha_i w_i + \delta_i w_{i-1} \rangle - \beta_m \langle v_{m-1}, w_i \rangle). \end{aligned}$$

For $i < m-1$ all the inner products vanish by induction hypothesis. For $i = m-1$

$$\begin{aligned} \langle v_{m+1}, w_{m-1} \rangle &= \delta_{m+1}^{-1} (\langle v_m, \beta_m w_m + \alpha_{m-1} w_{m-1} + \delta_{m-1} w_{m-2} \rangle \\ &\quad - \beta_m \langle v_{m-1}, w_{m-1} \rangle) \\ &= \delta_{m+1}^{-1} (\beta_m \langle v_m, w_m \rangle \\ &\quad + \underbrace{\alpha_{m-1} \langle v_m, w_{m-1} \rangle + \delta_{m-1} \langle v_m, w_{m-2} \rangle}_0 \\ &\quad - \beta_m \langle v_{m-1}, w_{m-1} \rangle) \\ &= \delta_{m+1}^{-1} (\underbrace{\beta_m \langle v_m, w_m \rangle}_1 - \underbrace{\beta_m \langle v_{m-1}, w_{m-1} \rangle}_1). \end{aligned}$$

Similar technique can be used to show that $w_{m+1}^T v_i = 0$ for $i = 1, \dots, m$. Finally, by construction, i.e. choice of $(\beta_{m+1}, \delta_{m+1})$ we have $\langle v_{m+1}, w_{m+1} \rangle = 1$.

Proof of $AV_m = V_m T_m + \delta_{m+1} v_{m+1} e_m^T$

$$T_m = \begin{pmatrix} \alpha_1 & \beta_2 & \beta_3 & & \\ \delta_2 & \alpha_2 & & & \\ & \ddots & \ddots & \ddots & \\ & & \delta_{m-1} & \alpha_{m-1} & \beta_m \\ & & & \delta_m & \alpha_m \end{pmatrix}.$$

Equality (18) can be shown for each column. Using Matlab notations: For $j \neq m$ [-10pt]

$$\begin{aligned} AV_m(:, j) &= V_m T(:, j) + \delta_{m+1} \times 0 \\ \Leftrightarrow Av_j &= \beta_j v_{j-1} + \alpha_j v_j + \delta_{j+1} v_{j+1} \end{aligned}$$

which corresponds to step 5 and 10 of the algorithm.

For $j = m$ [-10pt]

$$\begin{aligned} AV_m(:, j) &= V_m T(:, j) + \delta_{m+1} v_{m+1} \\ \Leftrightarrow Av_m &= (\beta_m v_{m-1} + \alpha_m v_m) + \delta_{m+1} v_{m+1} \end{aligned}$$

which also corresponds to step 5 and 10 of the algorithm for $j = m$.

Proof of $A^T W_m = W_m T_m^T + \beta_{m+1} w_{m+1} e_m^T$

$$T_m^T = \begin{pmatrix} \alpha_1 & \delta_2 & & & \\ \beta_2 & \alpha_2 & \delta_3 & & \\ & \ddots & \ddots & \ddots & \\ & & \beta_{m-1} & \alpha_{m-1} & \delta_m \\ & & & \beta_m & \alpha_m \end{pmatrix}$$

Equality (19) can be shown for each column. Using Matlab notations: For $j \neq m$ [-10pt]

$$\begin{aligned} A^T W_m(:, j) &= W_m T^T(:, j) + \beta_{m+1} \times 0 \\ \Leftrightarrow A^T w_j &= \delta_j w_{j-1} + \alpha_j w_j + \beta_{j+1} v_{j+1} \end{aligned}$$

which corresponds to step 6 and 9 of the algorithm.

For $j = m$ [-10pt]

$$\begin{aligned} A^T W_m(:, j) &= W_m T^T(:, j) + \beta_{m+1} w_{m+1} \\ \Leftrightarrow A^T w_m &= (\delta_m w_{m-1} + \alpha_m w_m) + \beta_{m+1} w_{m+1} \end{aligned}$$

which also corresponds to step 6 and 9 of the algorithm for $j = m$.

Proof of $W_m^T AV_m = T_m$

This is a direct consequence of Equality (18)

$$AV_m = V_m T_m + \delta_{m+1} v_{m+1} e_m^T,$$

and the biorthogonality of V_m and W_m

$$W_m^T V_m = I_m.$$

Two-Sided Lanczos algorithm for linear systems

This is a Ritz-Galerkin technique that seeks an approximate solution x_m of $Ax = b$ in the affine subspace $x_0 + \mathcal{K}(A, m, v_1)$ by imposing the condition

$$b - Ax_m \perp \mathcal{K}(A^T, m, w_1).$$

In that case, $b - Ax_m \perp \mathcal{K}(A^T, m, w_1)$ reads

$$\begin{aligned} W_m^T (b - A(x_0 + V_m y_m)) &= 0 \\ \Leftrightarrow W_m^T (r_0 - AV_m y_m) &= 0 \\ \Leftrightarrow W_m^T V_m (\beta e_1 - T_m y_m) &= 0 \quad \text{with } \beta = \|r_0\| \\ \Leftrightarrow y_m &= T_m^{-1} (\beta e_1) \end{aligned}$$

The dual problem solution

Notice that if we also had to solve $A^T x^* = b^*$ we could also use a Ritz-Galerkin technique that seeks an approximate solution x_m^* of $A^T x^* = b^*$ in the affine subspace $x_0^* + \mathcal{K}(A^T, m, w_1)$ by imposing the condition

$$b^* - A^T x_m^* \perp \mathcal{K}(A, m, v_1).$$

In that case, $b^* - A^T x_m^* \perp \mathcal{K}(A, m, v_1)$ reads

$$\begin{aligned} V_m^T (b^* - A^T (x_0^* + W_m y_m^*)) &= 0 \\ \Leftrightarrow V_m^T (r_0^* - A^T W_m y_m^*) &= 0 \\ \Leftrightarrow V_m^T W_m (\beta^* e_1 - T_m^T y_m^*) &= 0 \quad \text{with } \beta^* = \|r_0^*\| \\ \Leftrightarrow y_m^* &= T_m^{-T} (\beta^* e_1) \end{aligned}$$

The Two-Sided Lanczos algorithm for $Ax = b$

TWO-SIDED LANCZOS FOR LINEAR SYSTEMS [-10pt]

- 1: Compute $r_0 = b - Ax_0$, $\beta = \|r_0\|$
- 2: Run m steps of the non symmetric Lanczos; i.e.
- 3: Start with $v_1 = r_0/\beta$ and any vector w_1 such that $v_1^T w_1 = 1$
- 4: Generate the Lanczos vectors V_m , W_m and the tridiag. matrix T_m
- 5: Compute $y_m = T_m^{-1}(\beta e_1)$ and $x_m = x_0 + V_m y_m$

Notice that if a dual system had to be solved, choosing w_1 in the direction of $b^* - A^T x_0^*$ would enable the solution of the two linear systems at the same time:

$$y_m^* = T_m^{-T} (\beta^* e_1) \quad \text{and} \quad x_m^* = x_0^* + W_m y_m^*$$

where $\beta^* = \|r_0^*\|$.

Residual property

We also have

$$\begin{aligned} b - Ax_m &= b - A(x_0 + V_m y_m) \\ &= r_0 - AV_m y_m \\ &= V_m(\beta e_1) - (V_m T_m + \delta_{m+1} v_{m+1} e_m^T) y_m \\ &= \underbrace{V_m(\beta e_1 - T_m y_m)}_O - \delta_{m+1} v_{m+1} e_m^T y_m. \end{aligned}$$

Consequently

$$\|b - Ax_m\| = |\delta_{m+1} e_m^T y_m| \|v_{m+1}\|.$$

Similarly for the dual system we have

$$b^* - A^T x_m^* = -\beta_{m+1} w_{m+1} e_m^T y_m^*.$$

Consequently

$$\|b^* - A^T x_m^*\| = |\beta_{m+1} e_m^T y_m^*| \|w_{m+1}\|.$$

6.1 The BiCG algorithm

The BiCG variant

As for many other Krylov methods, we can exploit the structure of the projected matrix to derive an algorithm that enables us to perform an update of the solution at each step while avoiding to store the complete set of the basis vectors.

For the two-sided Lanczos algorithm, the projected matrix T_m is tridiagonal, we can compute an incremental LU factorization for it. This would lead to define the so-called Direct Two sided Lanczos method. However as CG was derived from Lanczos (more precisely D-Lanczos), we can directly derive its counterpart for the two-sided Lanczos.

Sketch of the derivation of BiCG

Analogy with D-Lanczos vs CG.

Symmetric Lanczos	Two-sided Lanczos
$AV_m = V_m T_m + \beta_{m+1} v_{m+1} e_m^T$	$AV_m = V_m T_m + \delta_{m+1} v_{m+1} e_m^T$ $A^T W_m = W_m T_m^T + \beta_{m+1} w_{m+1} e_m^T$
$\Downarrow$	$\Downarrow$
Incremental LU factorization of T_m	Incremental LU factorization of T_m
$\Downarrow$	$\Downarrow$
D-Lanczos	Direct two-sided Lanczos
$\Downarrow$	$\Downarrow$
Observe: r_i 's are orthogonal p_i 's are A-conjugate	Observe : r_i, r_j^* 's are orthogonal p_i, p_i^* 's are A-conjugate
$\Downarrow$	$\Downarrow$
Impose these constraints	Impose these constraints
$\Downarrow$	$\Downarrow$
CG	BiCG

Derivation of the BiCG algorithm

Proceeding in the same way as for the derivation of CG from symmetric Lanczos, we first write the LU decomposition of the unsymmetric tridiagonal T_m :

$$T_m = L_m U_m \quad T_m^T = U_m^T L_m^T.$$

For both the dual and primal linear systems we

$$P_m = V_m U_m^{-1} \quad P_m^* = W_m L_m^{-T}.$$

and compute the solutions

$$\begin{aligned} x_m &= x_0 + V_m T_m^{-1} (\beta e_1) & x_m^* &= x_0^* + W_m T_m^{-T} (\beta^* e_1) \\ &= x_0 + V_m U_m^{-1} L_m^{-1} (\beta e_1) & &= x_0^* + W_m L_m^{-T} U_m^{-T} (\beta^* e_1) \\ &= x_0 + P_m L_m^{-1} (\beta e_1) & &= x_0^* + P_m^* U_m^{-T} (\beta^* e_1) \end{aligned}$$

Observations

1. Orthogonality of the r_i, r_i^*

$$r_i = b - Ax_i = (-\delta_{i+1} e_i^T y_i) v_{i+1},$$

$$r_i^* = b^* - A^T x_i^* = (-\beta_{i+1} e_i^T y_i^*) w_{i+1},$$

is implied by the bi-orthogonality of v_i 's and w_i 's.

2. Conjugacy of the p_i, p_i^*

$$(P_m^*)^T A P_m = L_m^{-1} W_m^T A V_m U_m^{-1} = L_m^{-1} T_m U_m^{-1} = I_m.$$

The biconjugate gradient algorithm

BiCG ALGORITHM [-10pt]

- 1: $r_0 = b - Ax_0$
- 2: Choose r_0^* such that $r_0^T r_0^* \neq 0$
- 3: $p_0 = r_0, p_0^* = r_0^*$
- 4: **for** $j = 1, 2, \dots, m-1$ **do**
- 5: $\alpha_j = (r_j^T r_j^*) / ((p_j^*)^T A p_j)$
- 6: $x_{j+1} = x_j + \alpha_j p_j$
- 7: $r_{j+1} = r_j - \alpha_j A p_j$
- 8: $r_{j+1}^* = r_j^* - \alpha_j A^T p_j^*$

9: $\beta_j = (r_{j+1}^T r_{j+1}^*) / (r_j^T r_j^*)$
 10: $p_{j+1} = r_{j+1} + \beta_j p_j$
 11: $p_{j+1}^* = r_{j+1}^* + \beta_j p_j^*$
 12: **end for**

[C. Lanczos, *Solution of systems of linear equations by minimized iterations*, Journal of research of the national bureau of standards, 1952]

6.2 The QMR algorithm

The QMR algorithm

Based on the two-sided Lanczos algorithm a minimum norm residual approach can be developed.

$$AV_m = V_m T_m + \delta_{m+1} v_{m+1} e_m^T,$$

that can be re-written

$$AV_m = V_{m+1} \bar{T}_m,$$

where $\bar{T}_m$ is a $(m+1) \times m$ tridiagonal matrix

$$\bar{T}_m = \begin{pmatrix} T_m & \\ & \delta_{m+1} e_m^T \end{pmatrix}.$$

We can then derive a GMRES-like algorithm, if $v_1 = \beta^{-1} r_0$ and we look for a solution in the affine subspace $x_0 + \mathcal{K}(A, m, v_1)$

$$x_m = x_0 + V_m y.$$

[R. Freund and N. Nachtigal, *QMR: a quasi-minimal residual method for non-Hermitian linear systems*, Numerische Mathematik, 1991]

Its residual is given by

$$\begin{aligned}
 b - Ax_m &= b - A(x_0 + V_m y) \\
 &= r_0 - AV_m y \\
 &= \beta v_1 - V_{m+1} \bar{T}_m y \\
 &= V_{m+1} (\beta e_1 - \bar{T}_m y).
 \end{aligned}$$

and its norm

$$\|b - Ax_m\| = \|V_{m+1} (\beta e_1 - \bar{T}_m y)\|.$$

The Lanczos vectors v_i 's are not orthogonal. However, it seems reasonable to minimize

$$\|\beta e_1 - \bar{T}_m y\|$$

to compute the approximate solution x_m .

We can exploit the tridiagonal structure of $\bar{T}_m$ to derive an incremental QR factorization of $\bar{T}_m$ based on Givens rotations (similarly to DQGMRES for instance).

Relation between GMRES and QMR residuals

Proposition 6.17. *Assume that the Lanczos algorithm does not break down on or before step m and let V_{m+1} be the Lanczos basis obtained at step m . We have the following relation between $\|r_m^{QMR}\|$, the residual norm associated with the iterate computed by QMR at step m and $\|r_m^{GMRES}\|$, the residual norm associated with the iterate computed by GMRES:*

$$\|r_m^{QMR}\| \leq \kappa(V_{m+1}) \|r_m^{GMRES}\|.$$

Proof. In GMRES, $r_m^{GMRES} = r_0 + z_m$ with $z_m \in \mathcal{K}(A, m, r_0)$ where $z_m = AV_m^{GMRES} \tilde{y}_m^{GMRES} = AV_m^{QMR} y_m^{GMRES}$ since $\mathcal{K}(A, m, r_0) = \text{span}\{v_1^{QMR}, \dots, v_m^{QMR}\} = \text{span}\{v_1^{GMRES}, \dots, v_m^{GMRES}\}$.
 r_m^{GMRES} enables the minimization of

$$\|r_m^{GMRES}\| = \|V_{m+1}^{QMR} (\|r_0\| e_1 - \bar{T}_m y_m^{GMRES})\| = \|V_{m+1}^{GMRES} (\|r_0\| e_1 - \bar{H}_m \tilde{y}_m^{GMRES})\|;$$

In QMR, $r_m^{QMR} = r_0 - AV_m y_m^{QMR}$ where y_m^{QMR} enables the minimization of

$$\|\|r_0\| e_1 - \bar{T}_m y_m^{QMR}\|.$$

We have

$$\begin{aligned}
\|r_m^{QMR}\| &= \|V_{m+1}^{QMR} (\|r_0\|e_1 - \bar{T}_m y_m^{QMR})\| \\
&\leq \|V_{m+1}^{QMR}\| \|\|r_0\|e_1 - \bar{T}_m y_m^{QMR}\| \\
&\leq \|V_{m+1}^{QMR}\| \|\|r_0\|e_1 - \bar{T}_m y_m^{GMRES}\|, y_m^{QMR} \text{ minimizer} \\
&\leq \underbrace{\|V_{m+1}^{QMR}\| \sigma_{\min}^{-1}(V_{m+1}^{QMR})}_{\kappa(V_{m+1}^{QMR})} \underbrace{\|\|r_0\| (V_{m+1}^{QMR}(e_1 - \bar{T}_m y_m^{GMRES}))\|}_{r_m^{GMRES}}
\end{aligned}$$

because $\|x\| \leq \sigma_{\min}^{-1}(B)\|Bx\|$. □

6.3 Transpose free variant

6.3.1 The CGS algorithm

The Conjugate gradient squared (CGS)

As for any other Krylov method the residual computed by BiCG is such that

$$\exists \phi_j \in \mathbb{P}_j \text{ such that } r_j = \phi_j(A)r_0.$$

Similarly for the conjugate directions we have:

$$\exists \pi_j \in \mathbb{P}_j \text{ such that } p_j = \pi_j(A)p_0 = \pi_j(A)r_0,$$

because $P_m = V_m U_m^{-1}$ with U_m^{-1} upper triangular as U_m is upper triangular.

From the algorithm, we see that

$$\begin{aligned}
\phi_{j+1}(t) &= \phi_j(t) - \alpha_j t \pi_j(t), \\
\pi_{j+1}(t) &= \phi_{j+1}(t) + \beta_j \pi_j(t),
\end{aligned}$$

The Conjugate gradient squared (CGS)

Similarly r_j^* and p_j^* are generated using the same recurrences as r_j and p_j ; consequently the same polynomial but on A^T :

$$r_j^* = \phi_j(A^T)r_0^* \text{ and } p_j^* = \pi_j(A^T)p_0^*.$$

We observe that

$$\alpha_j = \frac{\langle \phi_j(A)r_0, \phi_j(A^T)r_0^* \rangle}{\langle A\pi_j(A)r_0, \pi_j(A^T)r_0^* \rangle} \stackrel{\phi(A^T)=(\phi(A))^T}{=} \frac{\langle \phi_j^2(A)r_0, r_0^* \rangle}{\langle A\pi_j^2(A)r_0, r_0^* \rangle}$$

and

$$\beta_j = \frac{\langle \phi_{j+1}(A)r_0, \phi_{j+1}(A^T)r_0^* \rangle}{\langle \phi_j(A)r_0, \phi_j(A^T)r_0^* \rangle} = \frac{\langle \phi_{j+1}^2(A)r_0, r_0^* \rangle}{\langle \phi_j^2(A)r_0, r_0^* \rangle}.$$

Governing idea

If we can define a recursion for $\phi_j^2(A)r_0$ and $\pi_j^2(A)r_0$ we could still compute α_j and β_j (without need anymore of product with A^T). We look for an algorithm that gives a sequence of iterates such that their residuals satisfy

$$r_j = \phi_j^2(A)r_0,$$

where $\phi_j(A)$ is the polynomial generated by BiCG. We have to define recurrences for the polynomial $\phi_j^2(A)$ and $\pi_j^2(A)$

Notice that if BiCG reduces the residual, the new algorithm should decrease it twice faster.

The derivation of the new algorithm is only based on polynomial and algebra manipulation. We have

$$\begin{aligned}
\phi_{j+1} &= \phi_j - \alpha_j t \pi_j, \\
\pi_{j+1} &= \phi_{j+1} + \beta_j \pi_j,
\end{aligned} \Rightarrow \begin{cases} \phi_{j+1}^2 = \phi_j^2 - 2\alpha_j t \phi_j \pi_j + \alpha_j^2 t^2 \pi_j^2, \\ \pi_{j+1}^2 = \phi_{j+1}^2 + 2\beta_j \phi_{j+1} \pi_j + \beta_j^2 \pi_j^2, \end{cases}$$

and

$$\begin{aligned}\phi_j \pi_j &= \phi_j (\phi_j + \beta_{j-1} \pi_{j-1}) = \phi_j^2 + \beta_{j-1} \phi_j \pi_{j-1}, \\ \phi_{j+1} \pi_j &= (\phi_j - \alpha_j t \pi_j) \pi_j = \phi_j \pi_j - \alpha_j t \pi_j^2 \\ &= (\phi_j^2 + \beta_{j-1} \phi_j \pi_{j-1}) - \alpha_j t \pi_j^2.\end{aligned}$$

Consequently we have 3 recursions that link ϕ_j^2 , π_j^2 and for $\phi_{j+1} \pi_j$, that are

$$\begin{cases} \phi_{j+1}^2 &= \phi_j^2 - \alpha_j t (2\phi_j^2 + 2\beta_j \phi_j \pi_{j-1} - \alpha_j t \pi_j^2), \\ \phi_{j+1} \pi_j &= (\phi_j^2 + \beta_{j-1} \phi_j \pi_{j-1}) - \alpha_j t \pi_j^2, \\ \pi_{j+1}^2 &= \phi_{j+1}^2 + 2\beta_j \phi_{j+1} \pi_j + \beta_j^2 \pi_j^2. \end{cases}$$

If we define

$$\begin{cases} r_j &= \phi_j^2(A) r_0, \\ p_j &= \pi_j^2(A) r_0, \\ q_j &= \phi_{j+1}(A) \pi_j(A) r_0, \end{cases}$$

this translates into the following vectors recursions

$$\begin{cases} r_{j+1} &= r_j - \alpha_j A (2r_j + 2\beta_{j-1} q_{j-1} - \alpha_j A p_j), \\ q_j &= r_j + \beta_{j-1} q_{j-1} - \alpha_j A p_j, \\ p_{j+1} &= r_{j+1} + 2\beta_j q_j + \beta_j^2 p_j. \end{cases}$$

To initiate the recursion for $q_j = r_j + \beta_{j-1} q_{j-1} - \alpha_j A p_j$ we have $q_{-1} = 0$ because

$$\begin{aligned}q_1 &= \phi_1(A) \pi_0(A) r_0 \\ &= \phi_1(A) r_0 \equiv r_1^{BiCG} = r_0 - \alpha_0 A p_0.\end{aligned}$$

We have the following algorithm

```

1:  $r_0 = b - Ax_0$ ,  $p_0 = 0, q_{-1} = 0$ 
2: for  $j = 1, 2, \dots$  do
3:    $\alpha_j = \langle r_j, r_0^* \rangle / \langle Ap_j, r_0^* \rangle$ 
4:    $d_j = 2r_j + 2\beta_{j-1} q_{j-1} - \alpha_j A p_j$ 
5:    $q_j = r_j + \beta_{j-1} q_{j-1} - \alpha_j A p_j$ 
6:    $r_{j+1} = r_j - \alpha_j A d_j$ 
7:    $x_{j+1} = x_j + \alpha_j d_j$ 
8:    $\beta_j = \langle r_{j+1}, r_0^* \rangle / \langle r_j, r_0^* \rangle$ 
9:    $p_{j+1} = r_{j+1} + 2\beta_j q_j + \beta_j^2 p_j$ 
10: end for
```

The above algorithm can be slightly simplified by introducing the auxiliary vector $u_j = r_j + \beta_{j-1} q_{j-1}$.

$$\begin{aligned}d_j &= u_j + q_j, \\ q_j &= u_j - \alpha_j A p_j, & \Rightarrow d_j \text{ is consequently no longer needed.} \\ p_{j+1} &= u_{j+1} + \beta_j (q_j + \beta_j p_j).\end{aligned}$$

CGS [-10pt]

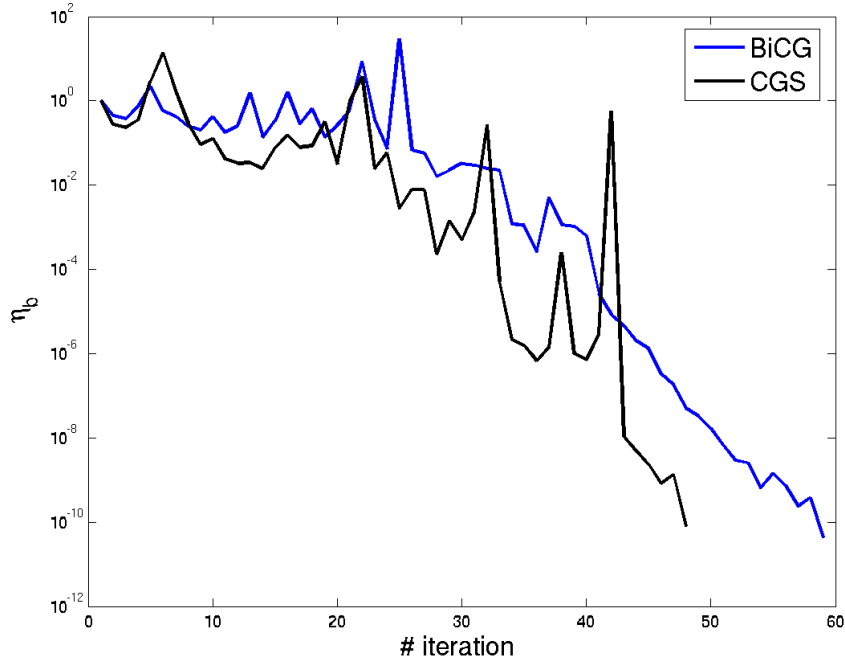
```

1:  $r_0 = b - Ax_0$ ,  $r_0^*$  arbitrary
2: for  $j = 1, 2, \dots$  do
3:    $\alpha_j = \langle r_j, r_0^* \rangle / \langle Ap_j, r_0^* \rangle$ 
4:    $q_j = u_j - \alpha_j A p_j$ 
5:    $r_{j+1} = r_j - \alpha_j A (u_j + q_j)$ 
6:    $x_{j+1} = x_j + \alpha_j (u_j + q_j)$ 
7:    $\beta_j = \langle r_{j+1}, r_0^* \rangle / \langle r_j, r_0^* \rangle$ 
8:    $u_{j+1} = r_{j+1} + \beta_j q_j$ 
9:    $p_{j+1} = u_{j+1} + \beta_j (q_j + \beta_j p_j)$ 
10: end for
```

Because $\phi_i^{CGS}(A) = (\phi_i^{BiCG}(A))^2$ CGS is expected to converge twice faster than BiCG.

[P. Sonneveld, *CGS: a fast Lanczos-type solver for nonsymmetric linear systems*, SIAM Journal on Scientific and Statistical Computing, 1989]

BiCG v.s. CGS: convergence behaviour



Squared polynomial effect

1. Faster convergence,
2. but larger peak/irregularity.

6.3.2 The BiCGStab algorithm

A smooth/stabilized BiCG variant: BiCGStab

Motivations: based on BiCG residual polynomial alleviate/smooth the peaks of CGS while still avoiding the use of A^T .

We look for an algorithm that gives a sequence of iterates such that their residuals satisfy

$$r_{j+1} = \psi_j(A)\phi_j(A)r_0,$$

where $\phi_j(t)$ is the polynomial generated by BiCG and $\psi(t)$ is a new recursively defined polynomial enabling a local control of the residual norm.

The polynomial is simply defined by

$$\psi_j(t) = (1 - \omega_j t) \psi_{j-1}(t).$$

The BiCGStab algorithm

In the BiCG algorithm we seek for polynomials for r_j and p_j .

$$\begin{aligned} r_{j+1}^{BiCG} &= \phi_j(A)r_0 = (r_j^{BiCG} - \alpha_j A p_j^{BiCG}), [\phi_{j+1}(t) = \phi_j(t) - \alpha_j t \pi_j(t)] \\ p_{j+1}^{BiCG} &= \pi_j(A)r_0 = r_{j+1}^{BiCG} + \beta_j p_j^{BiCG}; [\pi_{j+1}(t) = \phi_{j+1}(t) + \beta_j \pi_j(t)]. \end{aligned}$$

In the new algorithm they will be

$$\begin{aligned} r_{j+1} &= \psi_j(A)\phi_j(A)r_0 = (I - \omega_j A) \psi_{j-1}(A)\phi_j(A)r_0 \\ &= (I - \omega_j A) (r_j - \alpha_j A p_j) \\ p_{j+1} &= \psi_j(A)\pi_j(A)r_0 = r_{j+1} + \beta_j (I - \omega_j A) p_j. \end{aligned}$$

How to compute the α_j and β_j from BiCG polynomial without forming its sequence of iterates ?

We have

$$\beta_j = \frac{\langle \phi_{j+1}^2(A)r_0, r_0^* \rangle}{\langle \phi_j^2(A)r_0, r_0^* \rangle} = \frac{\rho_{j+1}^{BiCG}}{\rho_j^{BiCG}}.$$

but we don't have access to $\phi_j^2(A)$!

We can compute

$$\rho_j = \langle \phi_j(A)r_0, \psi_j(A^T)r_0^* \rangle = \langle \psi_j(A)\phi_j(A)r_0, r_0^* \rangle = \langle r_j, r_0^* \rangle$$

that might be related to ρ_j^{BiCG} using the following (exercise) properties of the BiCG algorithm iterates :

1. $\phi_j(A)r_0$ is orthogonal to all vectors $(A^T)^k r_0^*$ for $k < j$ (that is essentially related to the biorthogonality of $\mathcal{K}(A, m, r_0)$ and $\mathcal{K}(A^T, m, r_0^*)$),
2. Ap_j is orthogonal to $(A^T)^k r_0^*$ for $k < j$.

Residual properties

1. $\phi_j(A)r_0 \perp (A^T)^k r_0^*$ for $k < j$ $r_j = \phi_j(A)r_0$ is colinear to v_{j+1} , by bi-orthogonality of V_{j+1} and $W_{j+1} = \text{span}\{r_0^*, A^T r_0^*, \dots, (A^T)^j r_0^*\}$ the resultat holds.
2. Similarly $\alpha_j Ap_j = r_j - r_{j+1}$, r_j is colinear to v_{j+1} and r_{j+1} to v_{j+2} , because of the bi-orthogonality of V_{j+2} and W_{j+2} we have that r_j and r_{j+1} are orthogonal to $\text{span}\{w_1, \dots, w_j\} = \text{span}\{r_0^*, A^T r_0^*, \dots, (A^T)^{j-1}\}$

Calculation of β_j

Thanks to the first orthogonality property we have

$$\rho_j = \langle \phi_j(A)r_0, \psi_j(A^T)r_0^* \rangle = \langle \phi_j(A)r_0, \eta_j^{(j)} (A^T)^j r_0^* \rangle$$

where $\eta_j^{(j)}$ is the leading coefficient of the polynomial $\psi_j(t)$.

If we denote $\gamma_j^{(j)}$ the leading coefficient of the BiCG polynomial $\phi_j(t)$, we have

$$\rho_j = \langle \phi_j(A)r_0, \frac{\eta_j^{(j)}}{\gamma_j^{(j)}} \phi_j(A^T)r_0^* \rangle = \frac{\eta_j^{(j)}}{\gamma_j^{(j)}} \rho_j^{BiCG};$$

because the leading term $\frac{\eta_j^{(j)}}{\gamma_j^{(j)}} \phi_j(A^T)$ is $\eta_j^{(j)} (A^T)^j$. These leading coefficients can easily be computed from the recurrence relations defining the polynomial

$$\psi_j(t) = (1 - \omega_j t) \psi_{j-1}(t) \Rightarrow \eta_{j+1}^{(j+1)} = -\omega_j \eta_j^{(j)},$$

and

$$\left. \begin{array}{l} \phi_{j+1} = \phi_j - \alpha_j t \pi_j, \\ \pi_{j+1} = \phi_{j+1} + \beta_j \pi_j, \end{array} \right\} \Rightarrow \gamma_{j+1}^{(j+1)} = -\alpha_j \gamma_j^{(j)}.$$

Calculation of β_j

To summarize, we have

$$\left. \begin{array}{l} \frac{\rho_{j+1}}{\rho_{j+1}^{BiCG}} = \frac{\eta_{j+1}^{(j+1)}}{\gamma_{j+1}^{(j+1)}} \\ \frac{\eta_{j+1}^{(j+1)}}{\gamma_{j+1}^{(j+1)}} = \frac{\omega_j \eta_j^{(j)}}{\alpha_j \gamma_j^{(j)}} \end{array} \right\} \Rightarrow \frac{\rho_{j+1}}{\rho_{j+1}^{BiCG}} = \frac{\omega_j}{\alpha_j} \frac{\rho_j}{\rho_j^{BiCG}}.$$

Consequently

$$\beta_j = \frac{\rho_{j+1}^{BiCG}}{\rho_j^{BiCG}} = \frac{\rho_{j+1}}{\rho_j} \frac{\alpha_j}{\omega_j}.$$

Similar calculations can be developped to get $\alpha_j = \frac{\rho_j}{\langle Ap_j, r_0^* \rangle}$. This shows that the coefficients involved in the BiCG polynomial can effectively be recovered from quantities computed within the BiCGStab iterations.

Defining the free parameter ω_j

At each step the residual update is the following

$$r_{j+1} = (I - \omega_j A) (r_j - \alpha_j A p_j) = (I - \omega_j A) s_j.$$

In BiCGStab, ω_j is selected to minimize $\|r_{j+1}\|$:

$$\frac{\partial \|r_{j+1}\|}{\partial \omega} = 0 \Rightarrow \omega_j = \frac{s_j^T A s_j}{\|A s_j\|^2}.$$

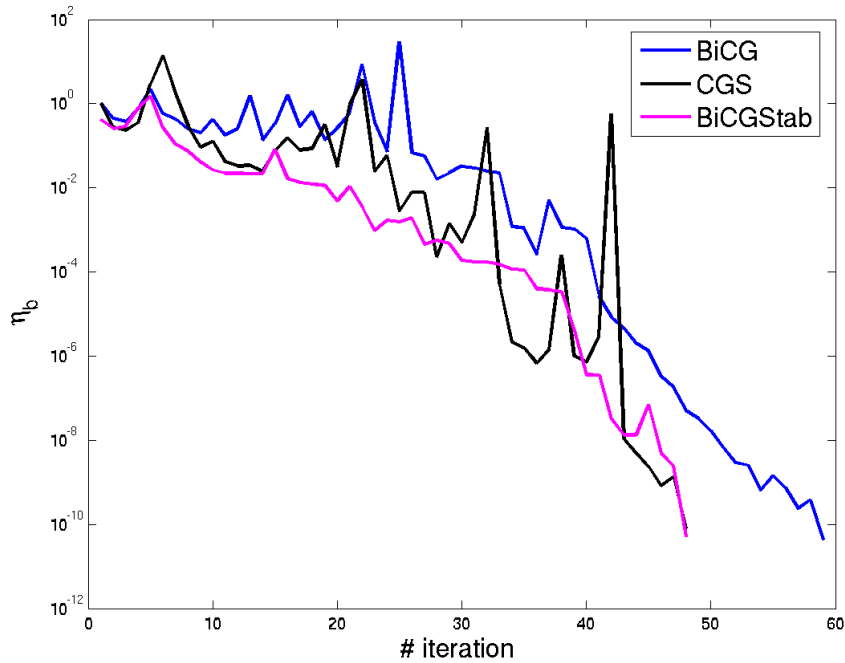
The BiCGStab algorithm

BiCGSTAB [-10pt]

- 1: Compute $r_0 = b - A x_0$, r_0^* arbitrary
- 2: $p_0 = r_0$
- 3: **for** $j = 1, 2, \dots$ **do**
- 4: $\alpha_j = \langle r_j, r_0^* \rangle / \langle A p_j, r_0^* \rangle$
- 5: $s_j = r_j - \alpha_j A p_j$
- 6: $\omega_j = \langle s_j, A s_j \rangle / \|A s_j\|^2$
- 7: $r_{j+1} = r_j - \alpha_j A p_j - \omega_j A s_j = s_j - \omega_j A s_j$
- 8: $x_{j+1} = x_j - \alpha_j p_j - \omega_j s_j$
- 9: $\beta_j = \frac{\alpha_j \langle r_{j+1}, r_0^* \rangle}{\langle r_j, r_0^* \rangle}$
- 10: $p_{j+1} = r_{j+1} + \beta_j (p_j - \omega_j A p_j)$
- 11: **end for**

[H.A. van der Vorst, *Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems*, SIAM J. Sci. Stat. Comput., 1992]

BiCG v.s. CGS: convergence behaviour



Observations

1. Converge as fast as CGS,
2. Smoother convergence.

7 Algebraic preconditioning techniques

7.1 Driving principles

Outline

- Some backgrounds
- Governing principles
- Preconditioner taxonomy and examples
- Spectral corrections

Some properties of Krylov solvers

- CG

$$\|x_k - x^*\|_A = \min_{p \in P_{k-1}, p(0)=1} \|p(A)(x_0 - x^*)\|_A.$$

where P_{k-1} is the set of polynomial of degree $(k-1)$.

- GMRES, MINRES

$$\|r_k\| = \min_{p \in P_{k-1}, p(0)=1} \|p(A)(r_0)\|_2.$$

- **Assumption:** if A diagonalizable with m distinct eigenvalues **then** CG, GMRES, MINRES converges in at most m steps. (Cayley-Hamilton theorem - minimal polynomial).
- **Assumption:** if r_0 has k components in the eigenbasis **then** CG, GMRES, MINRES converges in k steps.

Some properties of Krylov solvers

- Bound on the rate of convergence of CG

$$\|x_k - x^*\|_A \leq 2 \cdot \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k \|x_0 - x^*\|_A.$$

Driving principles to design preconditioners

Find a non-singular matrix M such that MA has “better” properties v.s. the convergence behaviour of the selected Krylov solver

- MA has less distinct eigenvalues,
- $MA \approx I$ in some sense.

The preconditioner constraints

The preconditioner should

- be cheap to compute and to store,
- be cheap to apply,
- ensure a fast convergence.

With a good preconditioner the solution time for the preconditioned system should be significantly less than for the unpreconditioned system.

7.2 Preconditioned CG

The particular case of CG

For CG let M be given in a factorized form (i.e. $M = CC^T$), then CG can be applied to

$$\tilde{A}\tilde{x} = \tilde{b},$$

with $\tilde{A} = C^T AC$, $C\tilde{x} = x$ and $\tilde{b} = C^T b$.

Let define:

$$\begin{aligned} x_k &= C\tilde{x}_k, \\ C\tilde{p}_k &= p_k, \\ \tilde{r}_k &= C^T r_k, \\ z_k &= CC^T r_k, \end{aligned}$$

Using $\tilde{A} = C^T AC$, $x_k = C\tilde{x}_k$, $p_k = C\tilde{p}_k$, $r_k = C^T \tilde{r}_k$ we can write the CG algorithm for both the preconditioned variables and the unpreconditioned ones.

Conjugate Gradient algorithm	
1. Compute $\tilde{r}_0 = \tilde{b} - \tilde{A}\tilde{x}_0$ and $\tilde{p}_0 = \tilde{r}_0$	
2. For k=0,2, ... Do	
3. $\alpha_k = \tilde{r}_k^T \tilde{r}_k / \tilde{p}_k^T \tilde{A} \tilde{p}_k$	$\alpha_k = r_k^T CC^T r_k / p_k^T A p_k$ $= r_k^T z_k / p_k^T A p_k$
4. $\tilde{x}_{k+1} = \tilde{x}_k + \alpha_k \tilde{p}_k$	$\xrightarrow{C} x_{k+1} = x_k + \alpha_k p_k$
5. $\tilde{r}_{k+1} = \tilde{r}_k - \alpha_k \tilde{A} \tilde{p}_k$	$\xrightarrow{C^T} r_{k+1} = r_k - \alpha_k A p_k$
6. $\beta_k = \tilde{r}_{k+1}^T \tilde{r}_{k+1} / \tilde{r}_k^T \tilde{r}_k$	$\beta_k = r_{k+1}^T CC^T r_{k+1} / r_k^T CC^T r_k$ $= z_{k+1}^T z_{k+1} / z_k^T z_k$
7. $\tilde{p}_{k+1} = \tilde{r}_{k+1} + \beta_k \tilde{p}_k$	$\xrightarrow{C} p_{k+1} = CC^T r_{k+1} + \beta_k p_k$ $= z_{k+1} + \beta_k p_k$
8. if $\tilde{x}_k$ accurate enough then stop	
9. EndDo	

Writing the algorithm only using the unpreconditioned variables leads to:

Preconditioned Conjugate Gradient algorithm	
1. Compute $r_0 = b - Ax_0$, $z_0 = Mr_0$ and $p_0 = r_0$	
2. For k=0,2, ... Do	
3. $\alpha_k = r_k^T z_k / p_k^T A p_k$	
4. $x_{k+1} = x_k + \alpha_k p_k$	
5. $r_{k+1} = r_k - \alpha_k A p_k$	
6. $z_{k+1} = M r_{k+1}$	
7. $\beta_k = r_{k+1}^T z_{k+1} / r_k^T z_k$	
8. $p_{k+1} = z_{k+1} + \beta_k p_k$	
9. if x_k accurate enough then stop	
10. EndDo	

A-norm minimization

The optimal property of CG (see Proposition 5.13) translates:

$$\|x_k - x^*\|_{C^T AC}$$

is minimal over $\mathcal{K}(C^T AC, C^T b, k)$. Notice, that

$$\begin{aligned} \|\tilde{x}_k - \tilde{x}^*\|_{C^T AC} &= (\tilde{x}_k - \tilde{x}^*)^T C^T AC (\tilde{x}_k - \tilde{x}^*) \\ &= (C(\tilde{x}_k - \tilde{x}^*))^T A (C(\tilde{x}_k - \tilde{x}^*)) \\ &= (x_k - x^*)^T A (x_k - x^*) \\ &= \|x_k - x^*\|_A. \end{aligned}$$

This shows that PCG also minimizes $\|x_k - x^*\|_A$ but on a different Krylov space than CG.

A-orthogonality of the p_k

The Proposition 5.12 shows that the vectors $\tilde{p}_k$ form an A-conjugate sets. This translates for $\ell \neq k$

$$\begin{aligned} 0 &= \tilde{p}_k^T C^T A C p_\ell \\ &= (C \tilde{p}_k)^T A (C \tilde{p}_\ell) \\ &= p_k^T A p_\ell, \end{aligned}$$

which shows that the p_k generated by PCG still form an A-conjugate sets.

Orthogonality of the r_k

By construction PCG builds a set of $\tilde{r}_k$ that are orthogonal. For the r_k this translates for $\ell \neq k$

$$\begin{aligned} 0 &= \tilde{r}_k^T r_\ell \\ &= (C^T r_k)^T C^T r_\ell \\ &= r_k^T C C^T r_\ell \\ &= r_k^T M r_\ell, \end{aligned}$$

which shows that the r_k generated by PCG are M -orthogonal.

7.3 Preconditioner taxonomy and location

MA has less distinct eigenvalues: an example

Let $\mathcal{A} = \begin{pmatrix} A & B^T \\ C & 0 \end{pmatrix}$ and $\mathcal{P} = \begin{pmatrix} A & 0 \\ 0 & CA^{-1}B^T \end{pmatrix}$.

Then $\mathcal{P}^{-1}\mathcal{A}$ has three distinct eigenvalues.

[Murphy, Golub, Wathen, SIAM SISC, 21 (6), 2000]

Proof

$$\mathcal{T} = \mathcal{P}^{-1}\mathcal{A} = \begin{pmatrix} I & A^{-1}B^T \\ (CA^{-1}B^T)^{-1}C & 0 \end{pmatrix}$$

and

$$\left(\mathcal{T} - \frac{1}{2}I\right)^2 = \begin{pmatrix} \frac{1}{4}I + A^{-1}B^T(CA^{-1}B^T)^{-1}C & 0 \\ 0 & \frac{5}{4}I \end{pmatrix}$$

where $A^{-1}B^T(CA^{-1}B^T)^{-1}C$ is a projection, so that

$$\left[\left(\mathcal{T} - \frac{1}{2}I\right)^2 - \frac{1}{4}I\right]^2 = \left[\left(\mathcal{T} - \frac{1}{2}I\right)^2 - \frac{1}{4}I\right],$$

which simplifies to

$$Q(\mathcal{T}) = \mathcal{T}(\mathcal{T} - I)(\mathcal{T}^2 - \mathcal{T} - I) = 0.$$

$\mathcal{T}$ has at most four distinct eigenvalues

$$0, 1, \frac{1}{2} \pm \frac{\sqrt{5}}{2}$$

because the minimal polynomial divides $Q(\mathcal{T})$.

Preconditioner taxonomy

There are two main classes of preconditioners

- **Implicit preconditioners:** approximate A with a matrix M such that solving the linear system $Mz = r$ is easy.
- **Explicit preconditioners:** approximate A^{-1} with a matrix M and just perform $z = Mr$.

The governing ideas in the design of the preconditioners are very similar to those followed to define iterative stationary schemes. Consequently, all the stationary methods can be used to define preconditioners.

Stationary methods

Let x_0 be given and $M \in \mathbb{R}^{n \times n}$ a nonsingular matrix, compute

$$x_k = x_{k-1} + M(b - Ax_{k-1}).$$

Note that $b - Ax_{k-1} = A(x^* - x_{k-1}) \Rightarrow$ the best M is A^{-1} .

The stationary scheme converges to $x^* = A^{-1}b$ for any x_0 iff $\rho(I - MA) < 1$, where $\rho(\cdot)$ denotes the spectral radius.

Let $A = L + D + U$

- $M = I$: Richardson method,
- $M = D^{-1}$: Jacobi method,
- $M = (L + D)^{-1}$: Gauss-Seidel method.

Notice that M has always a special structure and the inverse must never been explicitly computed ($z = B^{-1}y$ reads *solve the linear system $Bz = y$*).

Preconditioner location

Several possibilities exist to solve $Ax = b$:

- Left preconditioner

$$MAx = Mb.$$

- Right preconditioner

$$AMy = b \text{ with } x = My.$$

- Split preconditioner if $M = M_1M_2$

$$M_2AM_1y = M_2b \text{ with } x = M_1y.$$

Notice that the spectrum of MA , AM and M_2AM_1 are identical (for any matrices B and C , the eigenvalues of BC are the same as those of CB)

Preconditioner location v.s. stopping criterion

The stopping criterion are based on backward error

$$\eta_{A,b}^N = \frac{\|b - Ax\|}{\|A\|\|x\| + \|b\|} < \varepsilon \text{ or } \eta_b^N = \frac{\|b - Ax\|}{\|b\|} < \varepsilon.$$

In PCG we can still compute η . For GMRES, using a preconditioner means running GMRES on

$$\begin{array}{ccc} \text{Left precondition.} & \text{Right precondition.} & \text{Split precondition.} \\ MAx = Mb & AMy = b & M_2AM_1y = M_2b \end{array}$$

The free estimate of the residual norm of GMRES is associated with the preconditioned system.

Preconditioner location v.s. stopping criterion (cont)

	Left precondition.	Right precondition.	Split precondition.
$\eta_M(A, b)$	$\frac{\ MAx - Mb\ }{\ MA\ \ x\ + \ Mb\ }$	$\frac{\ AMy - b\ }{\ AM\ \ y\ + \ b\ }$	$\frac{\ M_2AM_1y - M_2b\ }{\ M_2AM_1\ \ y\ + \ M_2b\ }$
$\eta_M(b)$	$\frac{\ MAx - Mb\ }{\ Mb\ }$	$\frac{\ AMy - b\ }{\ b\ }$	$\frac{\ M_2AM_1y - M_2b\ }{\ M_2b\ }$

Using $\eta_M(b)$ for right preconditioned linear system will monitor the convergence as if no preconditioner was used.

7.4 Some classical algebraic preconditioners

Some classical algebraic preconditioner

- Incomplete factorization : IC , $ILU(p)$, $ILU(p, \tau)$
- SPAI (Sparse Approximate Inverse): compute the sparse approximate inverse by minimizing the Frobenius norm $\|MA - I\|_F$
- FSAI (Factorized Sparse Approximate inverse): compute the sparse approximate inverse of the Cholesky factor by minimizing the Frobenius norm $\|I - GL\|_F$
- AINV (Approximate Inverse): compute the sparse approximate inverse of the LDU or LDL^T factors using an incomplete biconjugation process

Incomplete factorizations

One variant of the LU factorization writes:

IKJ variant - Top looking variant	
1.	for $i = 2, \dots, n$ do
2.	for $k = 1, \dots, i - 1$ do
3.	$a_{i,k} = a_{i,k} / a_{k,k}$
4.	for $j = k + 1, \dots, n$ do
5.	$a_{i,j} = a_{i,j} - a_{i,k} * a_{k,j}$
6.	end for
7.	end for

Zero fill-in ILU - $ILU(0)$

Let denote $NZ(A)$ the set of (row,column) index of the nonzero entries of A .

$ILU(0)$	
1.	for $i = 2, \dots, n$ do
2.	for $k = 1, \dots, i - 1$ and $(i, k) \in NZ(A)$ do
3.	$a_{i,k} = a_{i,k} / a_{k,k}$
4.	for $j = k + 1, \dots, n$ and $(i, j) \in NZ(A)$ do
5.	$a_{i,j} = a_{i,j} - a_{i,k} * a_{k,j}$
6.	end for
7.	end for
8.	end for

Level of fill in ILU - $ILU(p)$

Definition 2. The initial level of fill of an entry $a_{i,j}$ is defined by:

$$lev(i, j) = \begin{cases} 0 & \text{if } a_{i,j} \neq 0 \text{ or } i = j, \\ \infty & \text{otherwise.} \end{cases}$$

Each time this entry is modified in line 5 of the LU top looking algorithm, its level fill is updated by

$$lev(i, j) = \min\{lev(i, j), lev(i, k) + lev(k, j) + 1\}. \quad (21)$$

A first example

$$A = \begin{pmatrix} x & x & x & x \\ x & x & & \\ x & & x & \\ x & & & x \end{pmatrix} \quad lev = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & \infty & \infty \\ 0 & \infty & 0 & \infty \\ 0 & \infty & \infty & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}.$$

ILU(p)	
1.	for all nonzero entries $a_{i,j}$ set $lev_{i,j}i = 0$
2.	for $i = 2, \dots, n$ do
3.	for $k = 1, \dots, i - 1$ and $lev_{i,k} \leq p$ do
4.	$a_{i,k} = a_{i,k}/a_{k,k}$
5.	for $j = k + 1, \dots, n$ do
6.	$a_{i,j} = a_{i,j} - a_{i,k} * a_{k,j}$
7.	$lev(i,j) = \min\{lev(i,j), lev(i,k) + lev(k,j) + 1\}$
8.	if $lev(i,j) > p$ then $a_{i,j} = 0$
9.	end for
10.	end for
11.	end for

Another example

$$\begin{pmatrix} \times & & & \times & & \times & & \times \\ & \times & & & \times & & & \\ & & \times & \times & & & & \times \\ \times & & & \times & \times & & & \\ & \times & \times & & \times & \times & & \\ & & \times & \times & & \times & \times & \\ \times & & & & & \times & \times & \\ & & & & \times & & \times & \end{pmatrix} \rightarrow \begin{pmatrix} 0 & \infty & \infty & 0 & \infty & 0 & \infty & 0 \\ \infty & 0 & \infty & \infty & 0 & \infty & \infty & \infty \\ \infty & 0 & 0 & \infty & 1 & \infty & 0 & \infty \\ 0 & \infty & \infty & 0 & 0 & 1 & \infty & 1 \\ \infty & 0 & 0 & \infty & 0 & 0 & \infty & 1 \\ \infty & \infty & 0 & 0 & 2 & 0 & 0 & 2 \\ 0 & \infty & \infty & 1 & 2 & 0 & 0 & 1 \\ \infty & \infty & \infty & \infty & 0 & 1 & 2 & 0 \end{pmatrix}$$

It may require to store many fill-ins that are small in absolute value.

ILU(p, τ) dual threshold strategy

- Fix a drop tolerance τ and a number of fill p to be allowed in each row of the incomplete LU factors. At each step of the elimination process, drop all fill-ins that are smaller than τ times the 2-norm of the current row; for all the remaining ones keep only the p largest.
- Trade-off between amount of fill-in (construction time and application time for the preconditioner) and decrease of number of iterations.

Incomplete factorization v.s. megaflop performance

The preconditioning step requires the solution of two sparse triangular factors that can lead to poor performance on vector computers due to the data dependencies. Special treatment can be implemented for structured matrices (FD matrices).

Computer	MFlops rate for PCG	MFlops rate for PCG with structure	MFlops rate for CG
NEX SX-3	60	607	1124
Cray C-90	56	444	737
RS 6000	19	18	21

The SPAI preconditioner

The idea is to compute the sparse approximate inverse as the matrix M which minimizes $\|I - MA\|_F$ (or $\|I - AM\|_F$ for right preconditioning) subject to certain sparsity constraints. The choice of the Frobenius norm is motivated by the identity:

$$\|I - AM\|_F^2 = \sum_{j=1}^n \|e_j - Am_{*,j}\|_2^2 \quad (22)$$

where e_j is the j th unit vector and $m_{*,j}$ is the column vector representing the j th row of M . Because of the sparsity constraint only a few rows and columns of A are used to compute each column of M . The least-squares problems can be solved efficiently with dense QR .

The SPAI preconditioner (cont)

- Embarrassingly parallel construction.
- Preconditioner application reduces to a sparse matrix vector product.
- For some applications the pattern of the inverse can be prescribed a priori (e.g. by considering the pattern of power of A , ParSails code by E. Chow).

The main difficulty consists in the determination of the sparsity pattern.

The Grote and Huckle SPAI Algorithm

```

1: for all column  $m_k$  of  $M$  do
2:   Choose the initial pattern  $\mathcal{J}$ 
3:   Determine the row indices  $\mathcal{I}$  of the corresponding nonzero entries and the  $QR$  decomposition of  $\hat{A} = A(\mathcal{I}, \mathcal{J})$ . Then
      compute the solution  $\hat{m}_j = \arg \min \|\hat{e}_j - \hat{A}\hat{m}_j\|$  and its residual  $r = \hat{e}_j - \hat{A}\hat{m}_j$ .
4:   while  $\|r\| > \varepsilon$  do
5:     Set  $\mathcal{L}$  be the set of  $\ell$  such that  $r(\ell) \neq 0$ 
6:     Set  $\tilde{\mathcal{J}} = \mathcal{L} \setminus \mathcal{J}$  the new candidates entries
7:     for all  $k \in \tilde{\mathcal{J}}$  do
8:       compute the norm  $\rho_k$  of the new residual  $\rho_k^2 = \|r\|^2 - \frac{(r^T A e_k)^2}{\|A e_k\|^2}$  obtained by solving the one-dimensional
          problem  $\min \|r - \mu A e_k\|$ . Only retain the most profitable indices.
9:     end for
10:    Determine the new set of row indices  $\tilde{\mathcal{I}}$  and update the  $QR$  factorization of  $A(\mathcal{I} \cup \tilde{\mathcal{I}}, \mathcal{J} \cup \tilde{\mathcal{J}})$ .
11:    Solve the new least-squares problem, the new associated residual and set  $\mathcal{I} = \mathcal{I} \cup \tilde{\mathcal{I}}$  and  $\mathcal{J} = \mathcal{J} \cup \tilde{\mathcal{J}}$ .
12:   end while
13: end for

```

The AINV method

Let $A = LDU$ then $A^{-1} = U^{-1}D^{-1}L^{-1} = ZD^{-1}W^T$ where $Z = U^{-1}$ and $W = L^{-T}$ are upper triangular.

$Z = [z_1, \dots, z_n]$ and $W = [w_1, \dots, w_n]$ are computed so that they are A -biconjugate, i.e. $w_i^T A v_j = 0$ for $i \neq j$.

$$W^T A Z = D = \begin{pmatrix} p_1 & 0 & \dots & 0 \\ 0 & p_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & p_n \end{pmatrix}$$

Using sparse approximation $\bar{Z} \approx Z$ and $\bar{W} \approx W$, the AINV preconditioner reads

$$M = \bar{Z} \bar{D}^{-1} \bar{W}^T \approx A^{-1}.$$

The A -biconjugation procedure

```

1:  $a_i = A(i, :)$ ,  $c_i = A(:, i)$  ( $1 \leq i \leq n$ )
2: Initialization:  $w_i^{(0)} = z_i^{(0)} = e_i$  ( $1 \leq i \leq n$ )
3: for  $i=1, \dots, n$  do
4:   for  $j=i+1, \dots, n$  do
5:      $p_j^{(i-1)} = a_i^T z_j^{(i-1)}$ ;  $q_j^{(i-1)} = c_i^T w_j^{(i-1)}$ 
6:   end for
7:   if  $i=n$  then break end if
8:   for  $j=i+1, \dots, n$  do
9:      $z_j^{(i)} = z_j^{(i-1)} - \frac{p_j^{(i-1)}}{p_i^{(i-1)}} z_i^{(i-1)}$ ;  $w_j^{(i)} = w_j^{(i-1)} - \frac{q_j^{(i-1)}}{q_i^{(i-1)}} w_i^{(i-1)}$ 
10:   end for
11: end for
12:  $z_i = z_i^{(i-1)}$ ;  $w_i = w_i^{(i-1)}$  and  $p_i = p_i^{(i-1)}$  ( $1 \leq i \leq n$ )

```

- If A is SPD, only Z needs to be computed. The procedure is a Gram-Schmidt orthogonalization with respect to the dot-product associated with A .

Preconditioner v.s. ordering

ILU and *AINV* preconditioner are sensitive to the ordering of the matrix A . Some “general” trends (experimental):

- *ILU*(p, τ): RCM is recommended even not necessary always the best.
- *AINV*: Minimum degree and Nested Dissection tend to perform better than RCM (sparser inverse factor). Reduced storage and better convergence.
- ND based ordering can be considered to parallelize some variants of these preconditioners.

Hybrid approaches

One route to the solution of large sparse linear systems in parallel scientific computing is the use of hybrid methods that combine direct and iterative methods. These techniques inherit the advantages of each approach, namely the limited amount of memory and easy parallelization for the iterative component and the numerical robustness of the direct part.

- Block preconditioners (block Jacobi, algebraic Schwarz variants, ...).
- Domain decomposition techniques.

7.5 Spectral preconditioners

Spectral preconditioners

Motivations

- The convergence of Krylov methods for solving the linear system often depends to a large extent on the eigenvalue distribution. In many cases, it is observed that “removing” the smallest eigenvalues can greatly improve the convergence
- Many preconditioners are able to cluster most of the eigenvalues close to one but still leave a few close to the origin
- “Moving” these eigenvalues by tuning the parameters that control these preconditioners is often difficult and might lead to very expensive preconditioners to set-up and to apply
- Performing a low rank update might enable to shift the smallest eigenvalues.

The SPD case - deflation approaches

Spectral decomposition of A

$$A[V_k V_k^\perp] = [V_k V_k^\perp]D.$$

- Splitting of the space

1. $x^{(0)} = V_k (V_k^T A V_k)^{-1} V_k^T b$,
2. Run $CG(A, b, x^{(0)})$.

- Deflated CG (augmented Krylov method)

$$\begin{cases} x^{(\ell)} \in x^{(0)} + \text{span}(V_k, r^{(0)}, \dots, r^{(\ell-1)}), \\ r^{(\ell)} \perp \text{span}(V_k, r^{(0)}, \dots, r^{(\ell-1)}). \end{cases}$$

- CG on projected system Let the projection $\mathcal{P} = I - A V_k (V_k^T A V_k)^{-1} V_k^T$ and write

$$x = (I - \mathcal{P}^T)x + \mathcal{P}^T x.$$

1. $(I - \mathcal{P}^T)x = V_k (V_k^T A V_k)^{-1} V_k^T b$,
2. Compute $\mathcal{P}^T x$ with CG by solving $\mathcal{P} A x = \mathcal{P} b$ exploiting

$$A \mathcal{P}^T = \mathcal{P} A.$$

Spectral preconditioner expression

Let $\{\lambda_1, \dots, \lambda_n\}$ be the set of eigenvalues of MA .

Suppose $MAV_k = V_k J_k$ where the eigenvalues of J_k are $\{\lambda_1, \dots, \lambda_k\}$. Let W be such that $A_c = W A V_k$ is nonsingular, $M_c = V_k A_c^{-1} W$ and

$$M_{ISLRU(k)} = M + M_c.$$

Then the eigenvalues of $M_{ISLRU}A$ (or AM_{ISLRU}) are

$$\begin{cases} \eta_i = \lambda_i & \text{if } i > k, \\ \eta_i = 1 + \lambda_i & \text{if } i \leq k. \end{cases}$$

- For SPD matrices, $W = V^T$.

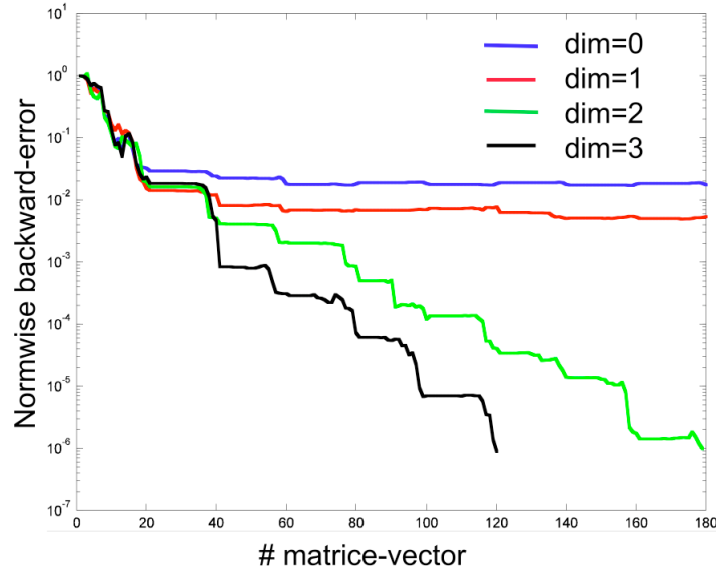
Spectral preconditioner expression (cont)

Similarly, $AMV = V J_k$ where the eigenvalues of J_k are $\{\lambda_1, \dots, \lambda_k\}$.

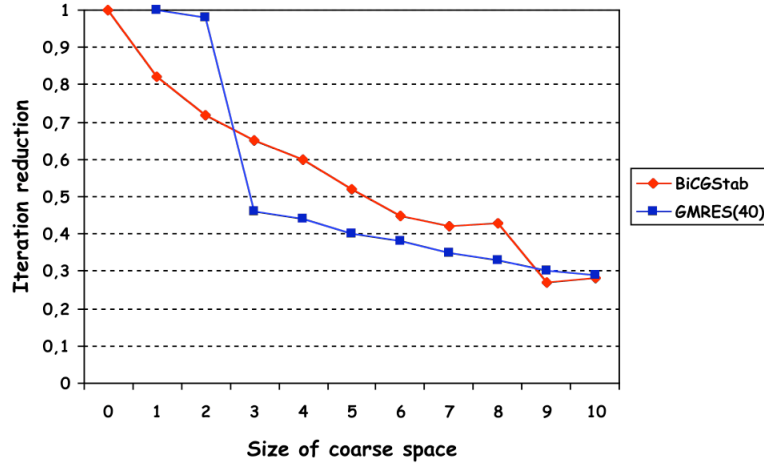
Let W be such that $A_c = W A M V_k$ is nonsingular, $M_c = M V_k A_c^{-1} W$ and $M_{ISLRU(k)} = M + M_c$. The eigenvalues of $AM_{ISLRU(k)}$ (or $M_{ISLRU(k)}A$) are

$$\begin{cases} \eta_i = \lambda_i & \text{if } i > k, \\ \eta_i = 1 + \lambda_i & \text{if } i \leq k. \end{cases}$$

Numerical behaviour



Numerical behaviour (cont)



Variable preconditioners

- Flexible variants of some Krylov solvers exist (FGMRES, GMRES*, Flexible CG) that enable to use different preconditioner at each step.
- For instance, the preconditioner can be computed by solving inaccurately a nearby problem

7.6 Flexible solver

Flexible preconditioner

One can be tempted to approximate the inverse of A by performing a few steps of a Krylov method. This does not lead to a linear operator (as the Krylov space depends on A but also on the right hand side). Consequently the preconditioner varies from one step to the next and we do no longer build a Krylov space. The solver should be revisited to handle this situation.

GMRES with right preconditioner

1. x_0 initial guess, $r_0 = b - Ax_0$, $\beta = \|r_0\|$ and $v_1 = r_0/\beta$
2. For $k=1,2, \dots$ Do
3. Compute $w_k = AMv_k$
4. For $i=1, \dots, j$, Do
5. $h_{i,k} = w_k^T v_i$
6. $w_k = w_k - h_{i,k}v_i$
7. EndDo
8. $h_{j+1,j} = \|w_k\|$
9. If $h_{j+1,j} = 0$ Goto 12
10. $v_{k+1} = w_k/h_{j+1,j}$
11. endDo
12. Set-up the $(m+1) \times m$ matrix $\bar{H}_m = (h_{i,j})_{1 \leq i \leq m+1, 1 \leq j \leq m}$
13. Compute, y_m the solution of $\|\beta e_1 - \bar{H}_m y\|_2$
14. Compute, $x_m = x_0 + MV_m y_m$

$$AMV_k = V_{k+1}\bar{H}_k$$

Flexible GMRES	
1.	x_0 initial guess, $r_0 = b - Ax_0$, $\beta = \ r_0\ $ and $v_1 = r_0/\beta$
2.	For $k=1,2, \dots$ Do
3.	Compute $z_k = M_k v_k$, $w_k = Az_k$
4.	For $i=1, \dots, j$, Do
5.	$h_{i,k} = w_k^T v_i$
6.	$w_k = w_k - h_{i,k} v_i$
7.	EndDo
8.	$h_{j+1,j} = \ w_k\ $
9.	If $h_{j+1,j} = 0$ Goto 12
10.	$v_{k+1} = w_k/h_{j+1,j}$
11.	endDo
12.	Set-up the $(m+1) \times m$ matrix $\bar{H}_m = (h_{i,j})_{1 \leq i \leq m+1, 1 \leq j \leq m}$
13.	Compute, y_m the solution of $\ \beta e_1 - \bar{H}_m y\ _2$
14.	Compute, $x_m = x_0 + Z_m y_m$

$$AZ_k = V_{k+1} \bar{H}_k$$

Proposition 7.18. *The approximate solution x_m obtained after m steps of FGMRES minimizes the residual norm $\|b - Ax_m\|_2$ over $x_0 + \text{span}\{Z_m\}$.*

Proof. From

$$AZ_m = V_{m+1} \bar{H}_m$$

we have

$$\begin{aligned}
\|b - Az\| &= \|b - A(x_0 + Z_m y)\| \\
&= \|r_0 - AZ_m y\| \\
&= \|\beta v_1 - V_{m+1} \bar{H}_m y\| \\
&= \|V_{m+1}(\beta e_1 - \bar{H}_m y)\| \\
&= \|\beta e_1 - \bar{H}_m y\|.
\end{aligned}$$

□

Proposition 7.19. *Assume that $\beta = \|r_0\| \neq 0$ and that $(j-1)$ steps of FGMRES have been successfully performed i.e. that $h_{i+1,i} \neq 0$ for $i < j$. In addition assume that the matrix H_j is nonsingular. Then x_j is exact iff $h_{j+1,j} = 0$.*

Proof. $\Rightarrow$ If $h_{j+1,j} = 0$ then $Az_j = V_j H_j$ and then

$$\|\beta v_1 - AZ_j y_j\| = \|\beta v_1 - V_j H_j y_j\| = \|\beta e_1 - H_j y_j\|.$$

If H_j is nonsingular, the minimum is obtained for $y_j - H_j^{-1}(\beta e_1)$ and its value is zero; i.e. x_j is exact.

$\Leftarrow$ We have

$$AZ_j = V_{j+1} \bar{H}_j = V_j H_j + \hat{v}_{j+1} e_j^T$$

where $\hat{v}_{j+1}$ is the new candidate to enlarge the basis before being normalized. If x_j is exact we have

$$0 = b - Ax_j = V_j[\beta e_1 - H_j y_j] + \hat{v}_{j+1} e_j^T y_j. \quad (23)$$

We show by contradiction $\hat{v}_{j+1} = 0$. Assume that $\hat{v}_{j+1} \neq 0$; since $[v_1, \dots, v_j]$ is an orthonormal basis it follows from (23) that $\beta e_1 - H_j y_j = 0$ and $e_j^T y_j = 0$ (i.e. the last entry of y_j is zero). A backsubstitution on the system $H_j y_j = \beta e_1$ starting from the last equation show that all the entries of y_j are zero. Because H_m is nonsingular this implies that $\beta = 0$ which contradicts the assumption. □

8 Bibliography

References

- [1] M. Benzi, *Preconditioning Techniques for Large Linear Systems: A Survey*, Journal of Computational Physics, 182, pp. 418-477, 2002.
- [2] A. Greenbaum, *Iterative methods for solving linear systems*, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1997.
- [3] G. Meurant, *Computer solution of large linear systems*, vol. 28 of Studies in Mathematics and its Applications, North-Holland Publishing Co., Amsterdam, 1999.
- [4] Y. Saad, *Iterative Methods for Sparse Linear Systems*, SIAM, Second edition, 2003.
- [5] H. A. van der Vorst, *Iterative Krylov methods for large linear systems*, Cambridge monographs on applied and computation mathematics, Cambridge University Press, Cambridge, 2003.